

Java面试题之Java集合框架篇（Java容器篇），29道Java集合框架八股文（7千字38张手绘图），面渣逆袭必看👉

👤 三分恶 📖 面渣逆袭 💎 面渣逆袭 🕒 约 16861 字 ⌚ 大约 56 分钟

7200 字 38 张手绘图，详解 29 道 Java 集合框架面试高频题（让天下没有难背的八股），面渣背会这些 Java 容器八股文，这次吊打面试官，我觉得稳了（手动 dog）。整理：沉默王二，戳[转载链接](#)，作者：三分恶，戳[原文链接](#)。

引言

1.说说有哪些常见的集合框架？

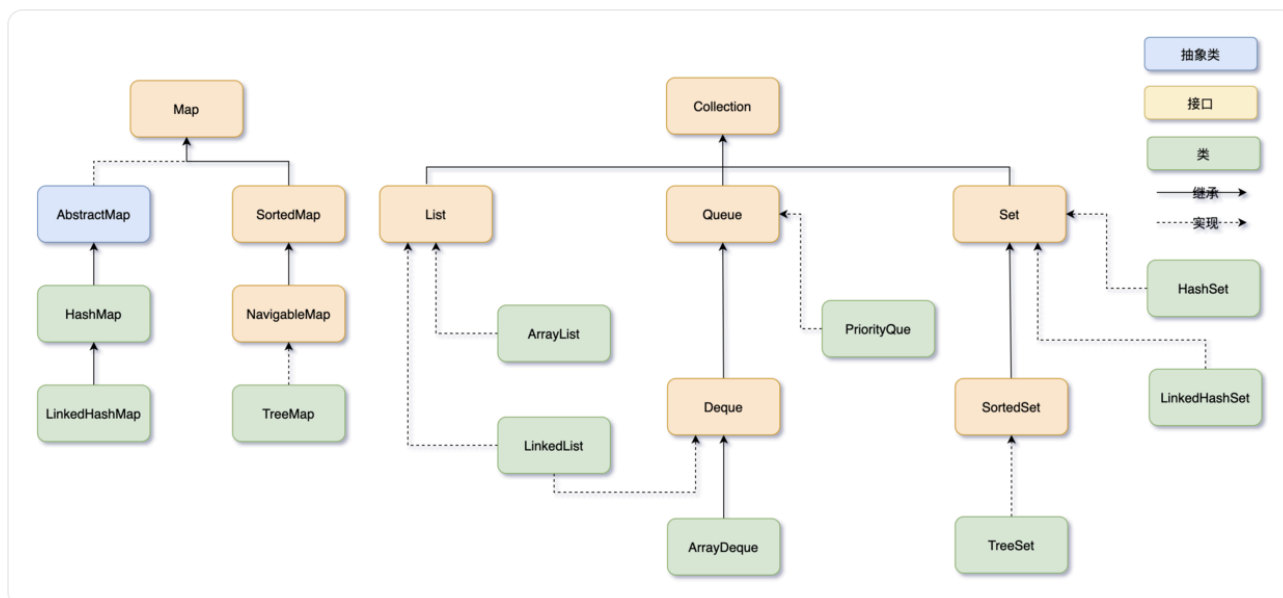
- 推荐阅读：[二哥的 Java 进阶之路：Java 集合框架](#)
- 推荐阅读：[阻塞队列 BlockingQueue](#)。

Java 集合框架可以分为两条大的支线：

①、Collection，主要由 List、Set、Queue 组成：

- List 代表有序、可重复的集合，典型代表就是封装了动态数组的 [ArrayList](#) 和封装了链表的 [LinkedList](#)；
- Set 代表无序、不可重复的集合，典型代表就是 HashSet 和 TreeSet；
- Queue 代表队列，典型代表就是双端队列 [ArrayDeque](#)，以及优先级队列 [PriorityQueue](#)。

②、Map，代表键值对的集合，典型代表就是 [HashMap](#)。



二哥的 Java 进阶之路：Java集合主要关系

概览图说明：

①、Collection 接口：最基本的集合框架表示方式，提供了添加、删除、清空等基本操作，它主要有三个子接口：

- List：一个有序的集合，可以包含重复的元素。实现类包括 ArrayList、LinkedList 等。
- Set：一个不包含重复元素的集合。实现类包括 HashSet、LinkedHashSet、TreeSet 等。
- Queue：一个用于保持元素队列的集合。实现类包括 PriorityQueue、ArrayDeque 等。

②、Map 接口：表示键值对的集合，一个键映射到一个值。键不能重复，每个键只能对应一个值。Map 接口的实现类包括 HashMap、LinkedHashMap、TreeMap 等。

集合框架位于 java.util 包下，该包提供了两个常用的工具类：

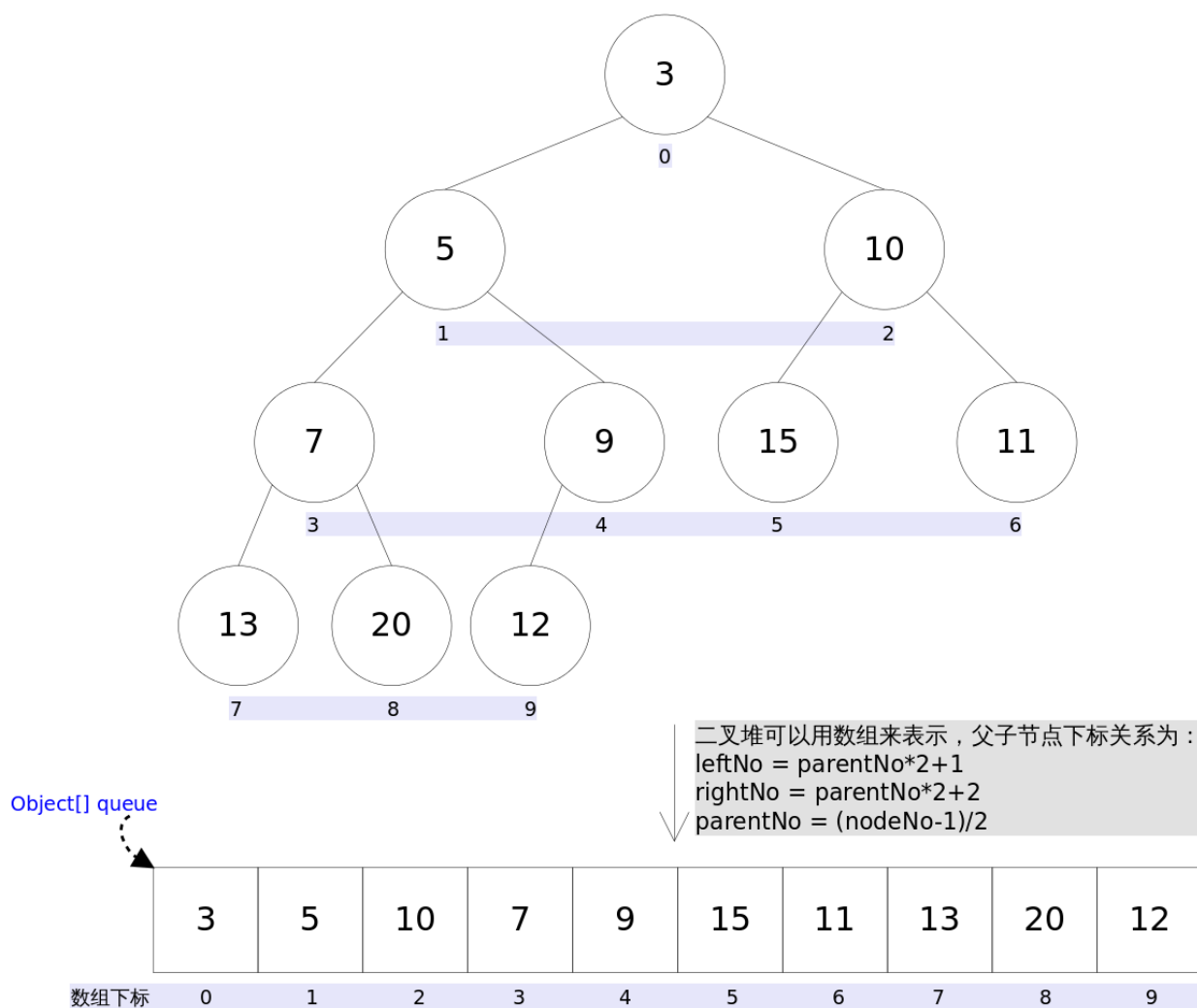
- [Collections](#)：提供了一些对集合进行排序、二分查找、同步的静态方法。
- [Arrays](#)：提供了一些对数组进行排序、打印、和 List 进行转换的静态方法。

简单介绍一下队列 Queue

Java 中的队列主要通过 java.util.Queue 接口和 java.util.concurrent.BlockingQueue 两个接口来实现。

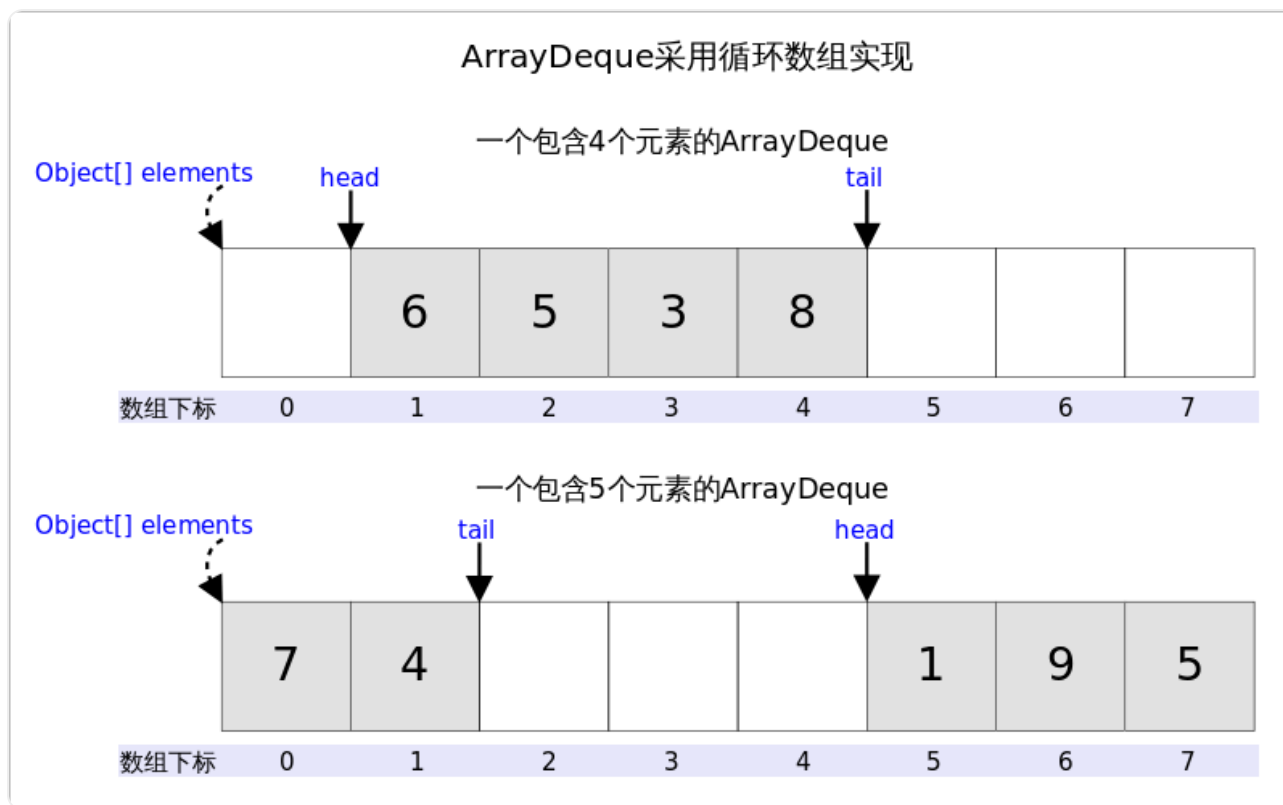
PriorityQueue 是一个基于优先级堆的无界队列，它的元素按照自然顺序排序或者 Comparator 进行排序。

PriorityQueue通过用数组表示的小顶堆实现



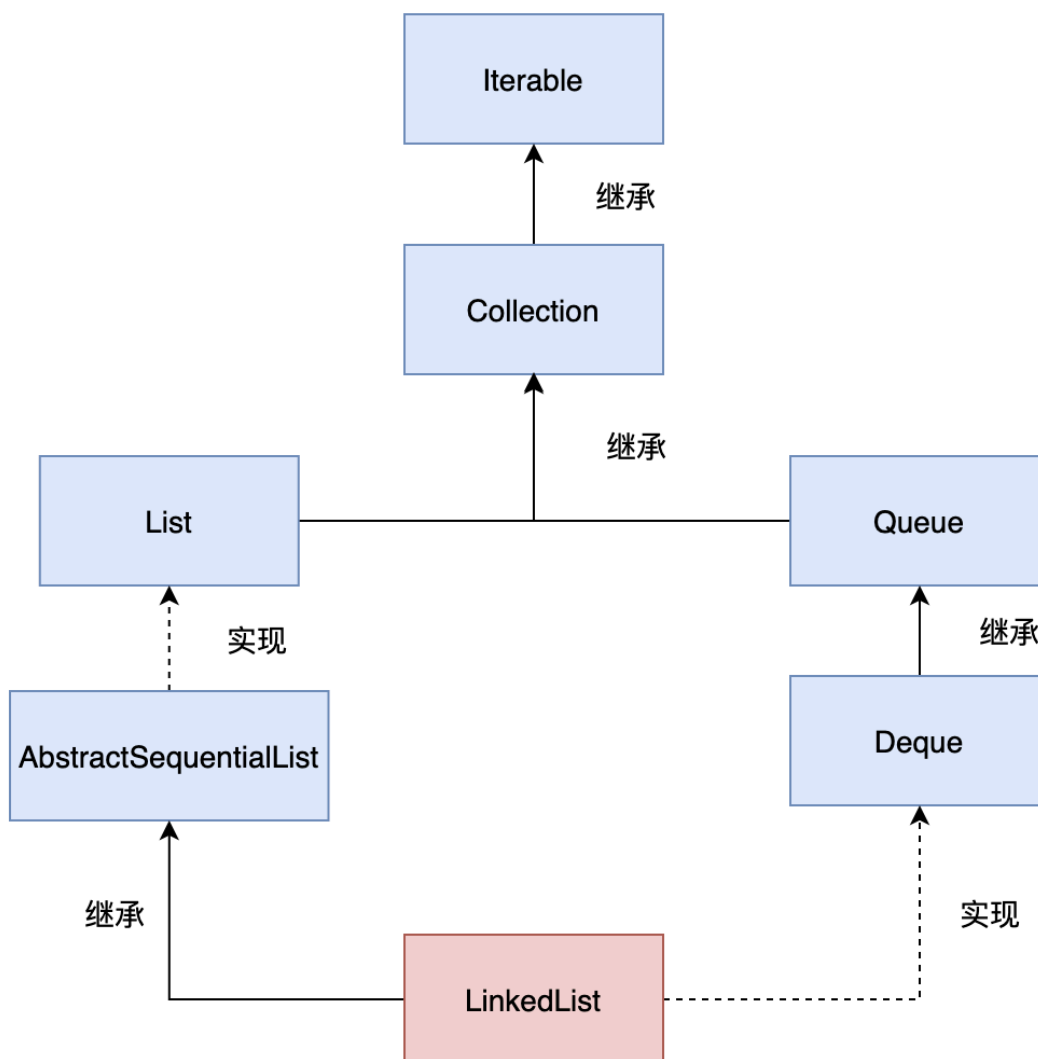
李豪：优先级队列

ArrayDeque 是一个基于数组的双端队列，可以在两端插入和删除元素。



李豪：双端队列

还有一个大家可能忽略的队列，那就是 LinkedList，它既可以当作 List 使用，也可以当作 Queue 使用。



二哥的 Java 进阶之路

用过哪些集合类，它们的优劣？

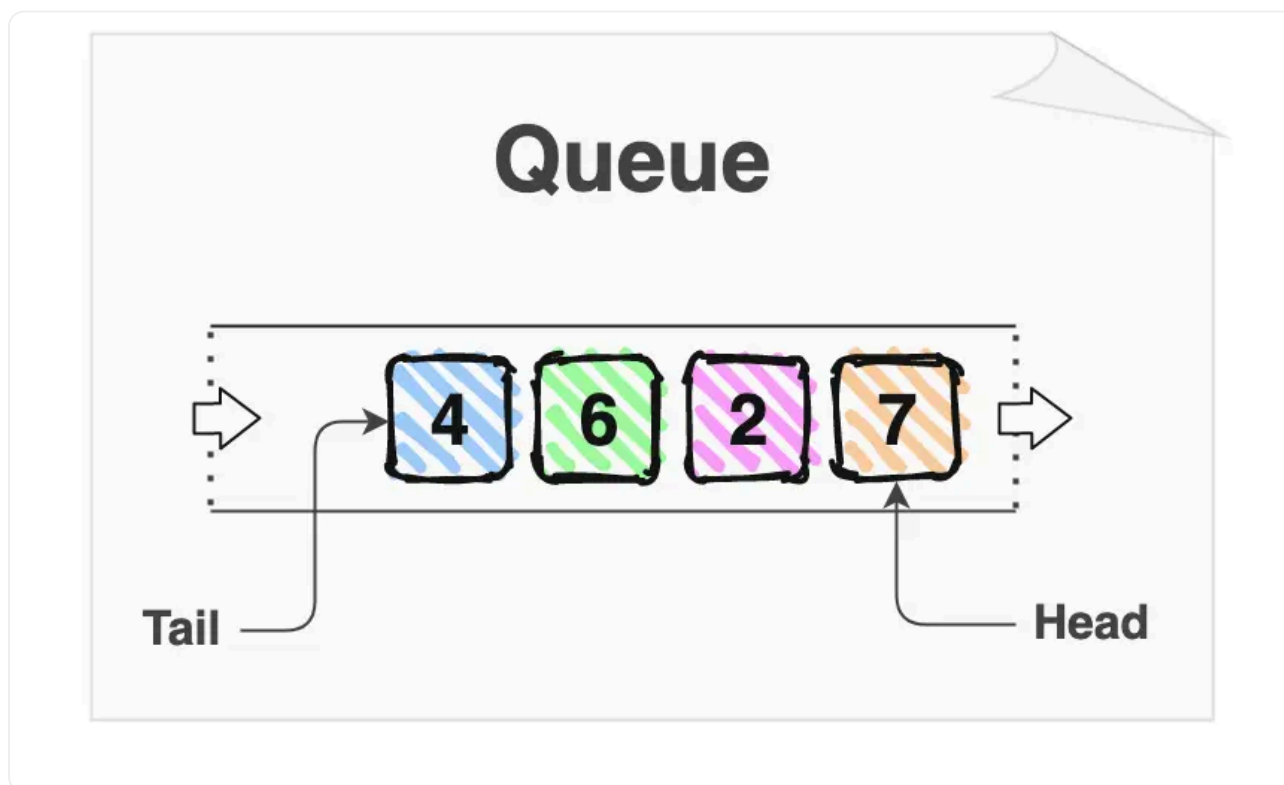
在 Java 中，常用的集合类有 `ArrayList`、`LinkedList`、`HashMap`、`LinkedHashMap` 等。

1. `ArrayList`：`ArrayList` 可以看作是一个动态数组，它可以在运行时动态扩容。优点是访问速度快，可以通过索引直接查到元素。缺点是插入和删除元素可能需要移动元素，效率就会降低。
2. `LinkedList`：`LinkedList` 是一个双向链表，它适合频繁的插入和删除操作。优点是插入和删除元素的时候只需要改变节点的前后指针，缺点是访问元素时需要遍历链表。

3. HashMap: HashMap 是一个基于哈希表的键值对集合。优点是插入、删除和查找元素的速度都很快。缺点是它不保留键值对的插入顺序。
4. LinkedHashMap: LinkedHashMap 在 HashMap 的基础上增加了一个双向链表来保持键值对的插入顺序。

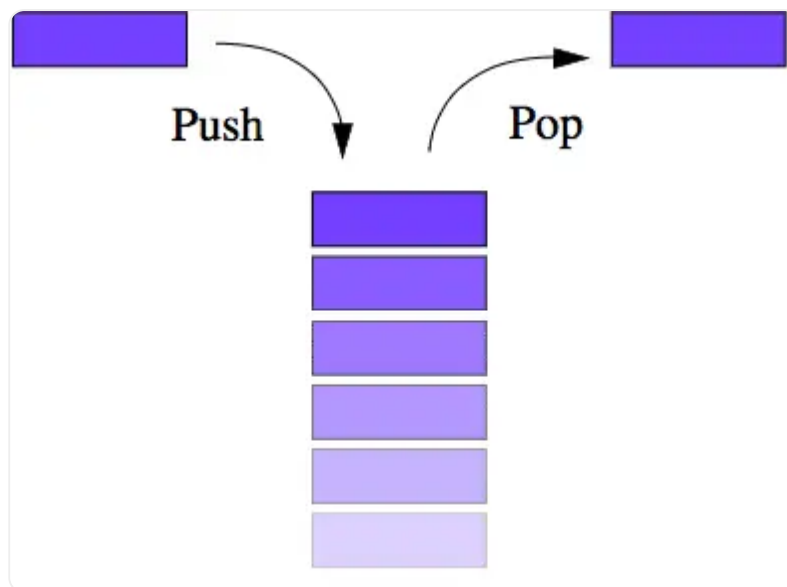
队列和栈的区别了解吗？

队列是一种先进先出（FIFO, First-In-First-Out）的数据结构。在队列中，第一个加入队列的元素会是第一个被移除的。队列常用于处理按顺序来的任务。



疯狂的技术宅：队列

栈是一种后进先出（LIFO, Last-In-First-Out）的数据结构。在这种结构中，最后一个加入栈的元素会是第一个被移除的。这种特性使得栈非常适合于那些需要访问最新添加的数据元素的场合。



Wang Wei：栈

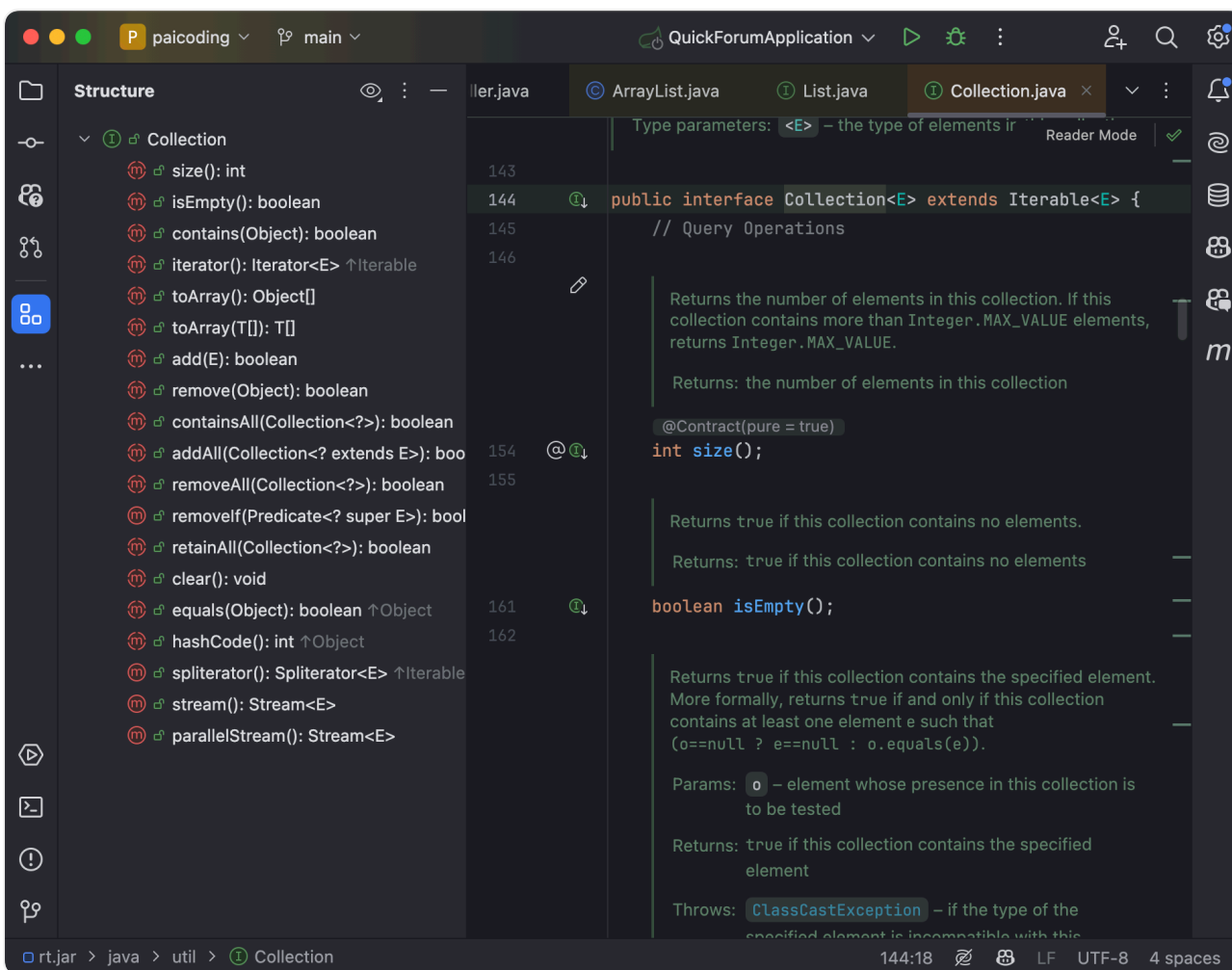
哪些是线程安全的？

像 `Vector`、`Hashtable`、`ConcurrentHashMap`、`CopyOnWriteArrayList`、`ConcurrentLinkedQueue`、`ArrayBlockingQueue`、`LinkedBlockingQueue` 这些都是线程安全的。

Java 集合用过哪些？`Collection` 继承了哪些接口？

最经常用的就是封装了动态数组的 `ArrayList` 和封装了链表的 `LinkedList`；以及键值对 `HashMap`。

`Collection` 继承了 `Iterable` 接口，这意味着所有实现 `Collection` 接口的类都必须实现 `iterator()` 方法，之后就可以使用增强型 `for` 循环遍历集合中的元素了。



二哥的 Java 进阶之路：Collection源码

1. [Java 面试指南（付费）](#) 收录的用友金融一面原题：你了解哪些集合框架？
2. [Java 面试指南（付费）](#) 收录的华为一面原题：说下 Java 容器和 HashMap
3. [Java 面试指南（付费）](#) 收录的小米暑期实习同学 E 一面面试原题：你了解哪些集合？
4. [Java 面试指南（付费）](#) 收录的美团面经同学 16 暑期实习一面面试原题：知道哪些集合，讲讲 HashMap 和 TreeMap 的区别，讲讲两者应用场景的区别；讲一下有哪些队列，阻塞队列的阻塞是什么含义？
5. [Java 面试指南（付费）](#) 收录的农业银行面经同学 7 Java 后端面试原题：用过哪些集合类，它们的优劣
6. [Java 面试指南（付费）](#) 收录的华为 OD 面经同学 1 一面面试原题：队列和栈的区别了解吗？
7. [Java 面试指南（付费）](#) 收录的农业银行同学 1 面试原题：阻塞队列的实现方式
8. [Java 面试指南（付费）](#) 收录的小公司面经合集同学 1 Java 后端面试原题：Java 容器有哪些？List、Set 还有 Map 的区别？
9. [Java 面试指南（付费）](#) 收录的 360 面经同学 3 Java 后端技术一面面试原题：java 有哪些集合

10. [Java 面试指南（付费）](#) 收录的华为面经同学 11 面试原题：java 中的集合类型？哪些是线程安全的？
11. [Java 面试指南（付费）](#) 收录的招商银行面经同学 6 招银网络科技面试原题：Java 集合有哪些？
12. [Java 面试指南（付费）](#) 收录的用友面试原题：集合容器能列举几个吗？

List

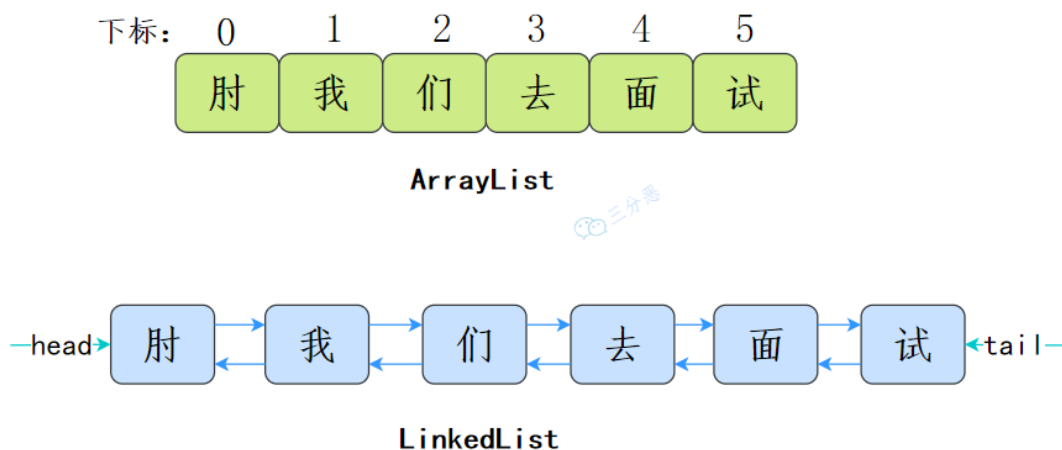
2.ArrayList 和 LinkedList 有什么区别？

推荐阅读：[二哥的 Java 进阶之路：ArrayList 和 LinkedList](#)

ArrayList 和 LinkedList 的区别主要体现在数据结构、用途、是否支持随机访问、内存占用等方面。

数据结构有什么不同？

- ArrayList 基于数组实现
- LinkedList 基于链表实现



三分恶面渣逆袭：ArrayList和LinkedList的数据结构

用途有什么不同？

多数情况下，ArrayList 更利于查找，LinkedList 更利于增删

①、由于 `ArrayList` 是基于数组实现的，所以 `get(int index)` 可以直接通过数组下标获取，时间复杂度是 $O(1)$ ；`LinkedList` 是基于链表实现的，`get(int index)` 需要遍历链表，时间复杂度是 $O(n)$ 。

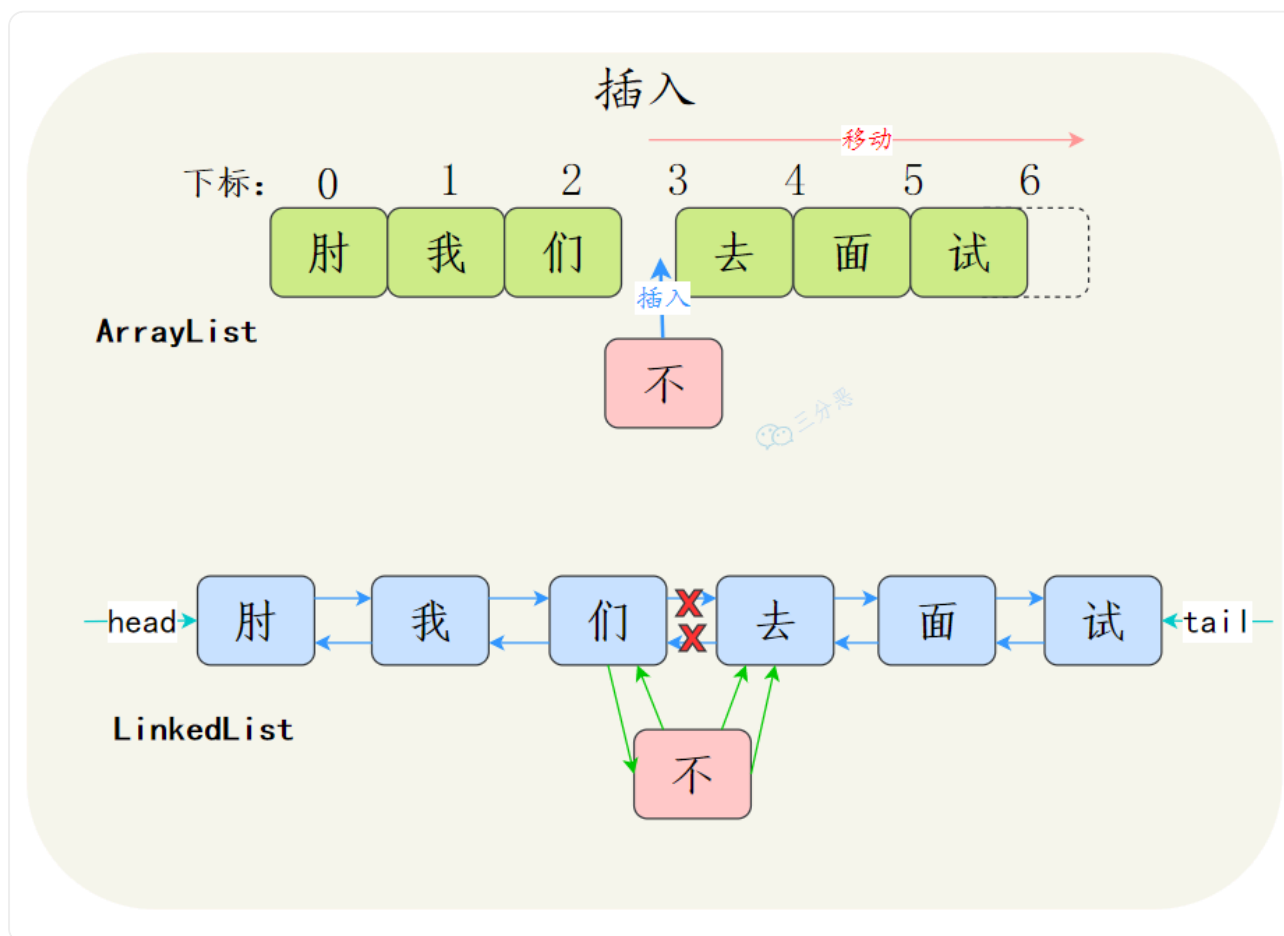
当然，`get(E element)` 这种查找，两种集合都需要遍历通过 `equals` 比较获取元素，所以时间复杂度都是 $O(n)$ 。

②、`ArrayList` 如果增删的是数组的尾部，直接插入或者删除就可以了，时间复杂度是 $O(1)$ ；如果 `add` 的时候涉及到扩容，时间复杂度会提升到 $O(n)$ 。

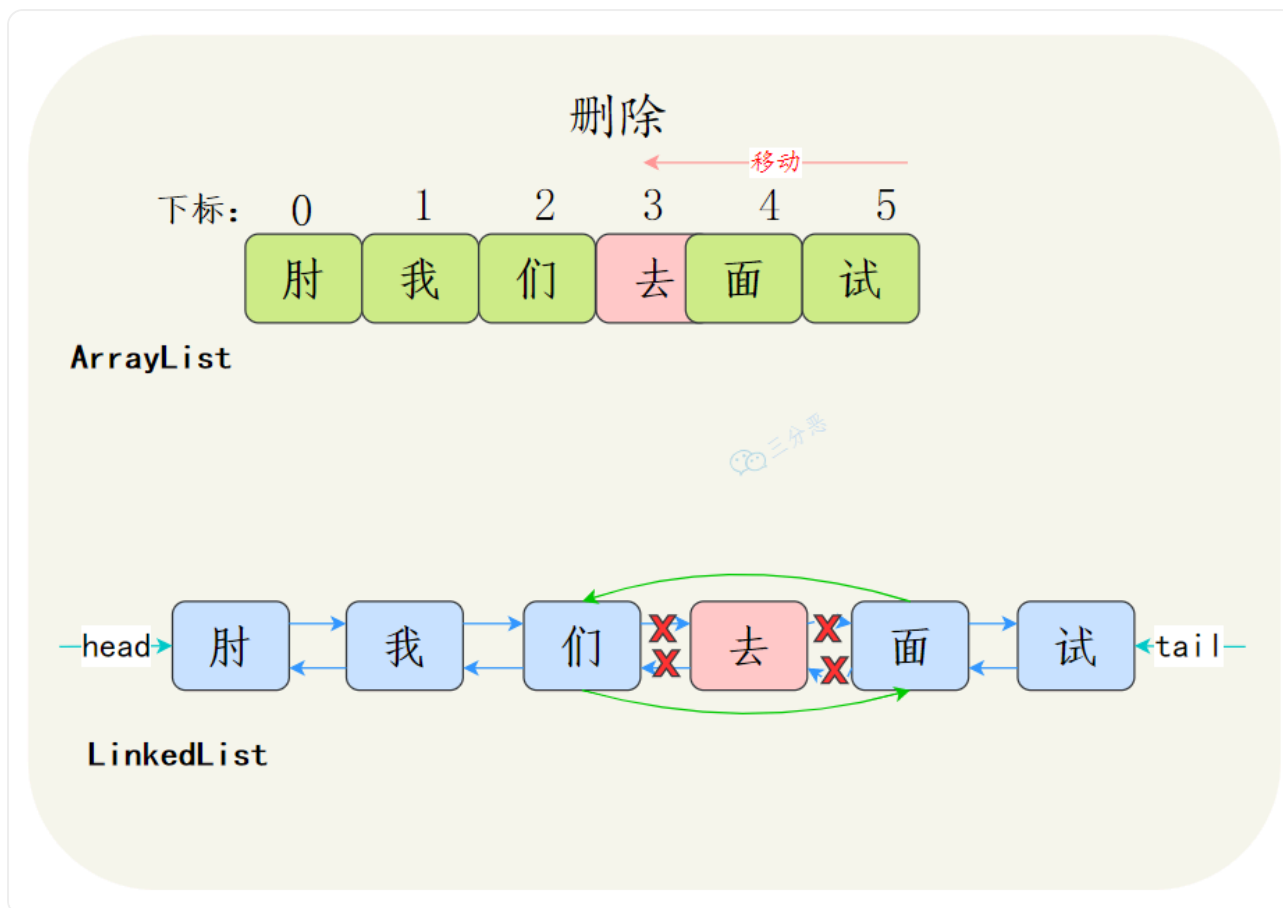
但如果插入的是中间的位置，就需要把插入位置后的元素向前或者向后移动，甚至还有可能触发扩容，效率就会低很多， $O(n)$ 。

`LinkedList` 因为是链表结构，插入和删除只需要改变前置节点、后置节点和插入节点的引用就行了，不需要移动元素。

如果是在链表的头部插入或者删除，时间复杂度是 $O(1)$ ；如果是在链表的中间插入或者删除，时间复杂度是 $O(n)$ ，因为需要遍历链表找到插入位置；如果是在链表的尾部插入或者删除，时间复杂度是 $O(1)$ 。



三分恶面渣逆袭：ArrayList和LinkedList中间插入

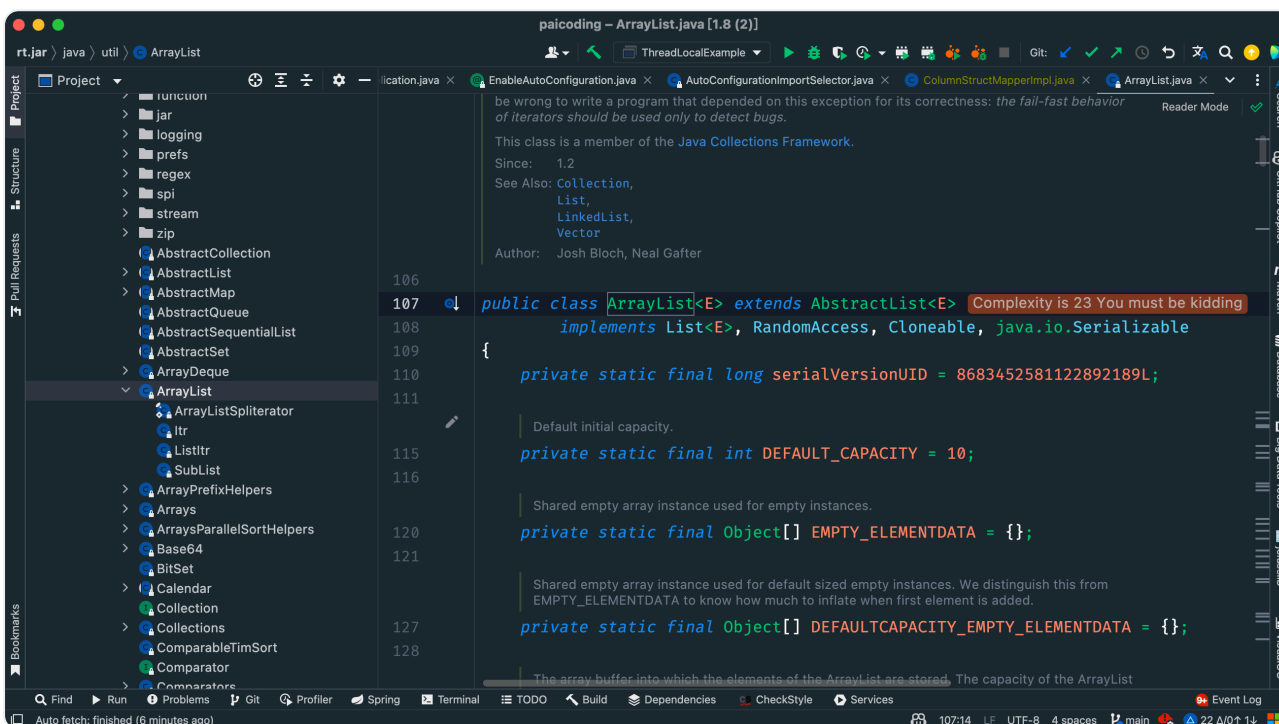


三分恶面渣逆袭：ArrayList和LinkedList中间删除

注意，这里有个陷阱，LinkedList 更利于增删不是体现在时间复杂度上，因为二者增删的时间复杂度都是 $O(n)$ ，都需要遍历列表；而是体现在增删的效率上，因为 LinkedList 的增删只需要改变引用，而 ArrayList 的增删可能需要移动元素。

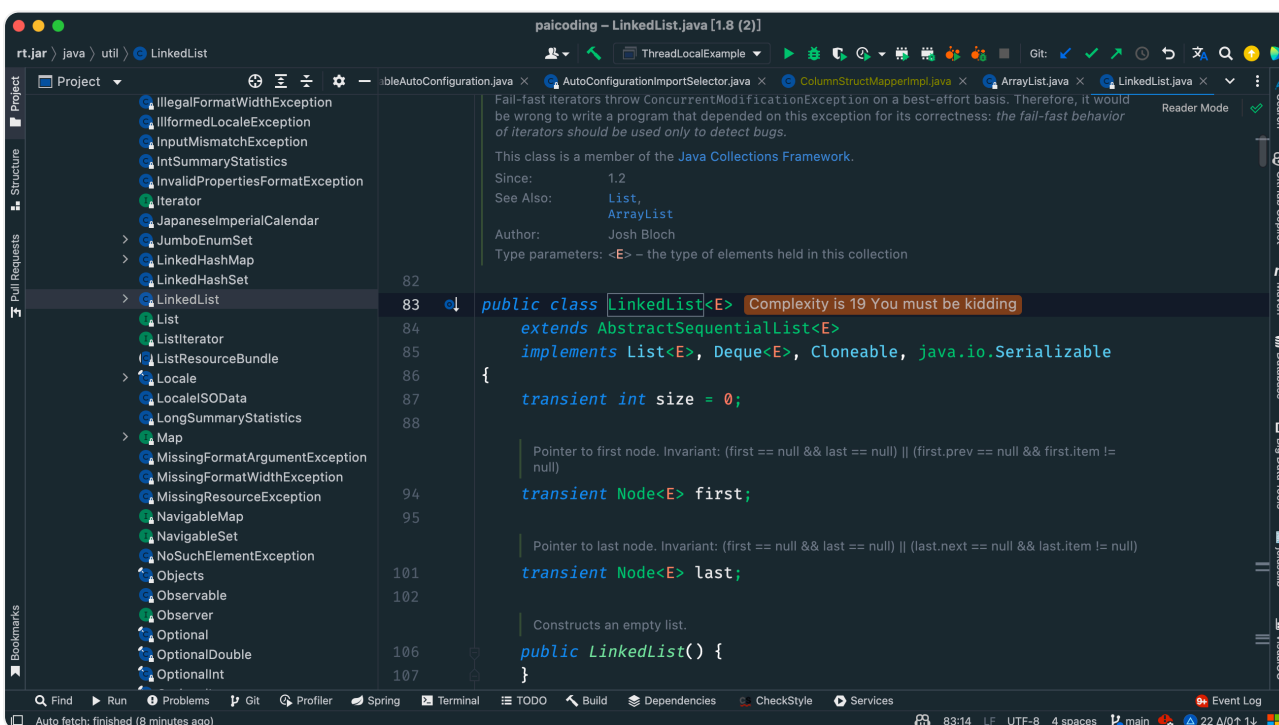
是否支持随机访问？

①、ArrayList 是基于数组的，也实现了 RandomAccess 接口，所以它支持随机访问，可以通过下标直接获取元素。



二哥的 Java 进阶之路：ArrayList

②、LinkedList 是基于链表的，所以它没法根据下标直接获取元素，不支持随机访问，所以它也没有实现 RandomAccess 接口。



二哥的 Java 进阶之路：LinkedList

内存占用有何不同？

ArrayList 是基于数组的，是一块连续的内存空间，所以它的内存占用是比较紧凑的；但如果涉及到扩容，就会重新分配内存，空间是原来的 1.5 倍，存在一定的空间浪费。

```
private void grow(int minCapacity) {  
    // overflow-conscious code  
    int oldCapacity = elementData.length;  
    int newCapacity = oldCapacity + (oldCapacity >> 1);  
    if (newCapacity - minCapacity < 0)  
        newCapacity = minCapacity;  
    if (newCapacity - MAX_ARRAY_SIZE > 0)  
        newCapacity = hugeCapacity(minCapacity);  
    // minCapacity is usually close to size, so this is a win:  
    elementData = Arrays.copyOf(elementData, newCapacity);  
}
```

二哥的 Java 进阶之路：ArrayList的扩容

LinkedList 是基于链表的，每个节点都有一个指向下一个节点和上一个节点的引用，于是每个节点占用的内存空间稍微大一点。

使用场景有什么不同？

ArrayList 适用于：

- 随机访问频繁：需要频繁通过索引访问元素的场景。
- 读取操作远多于写入操作：如存储不经常改变的列表。
- 末尾添加元素：需要频繁在列表末尾添加元素的场景。

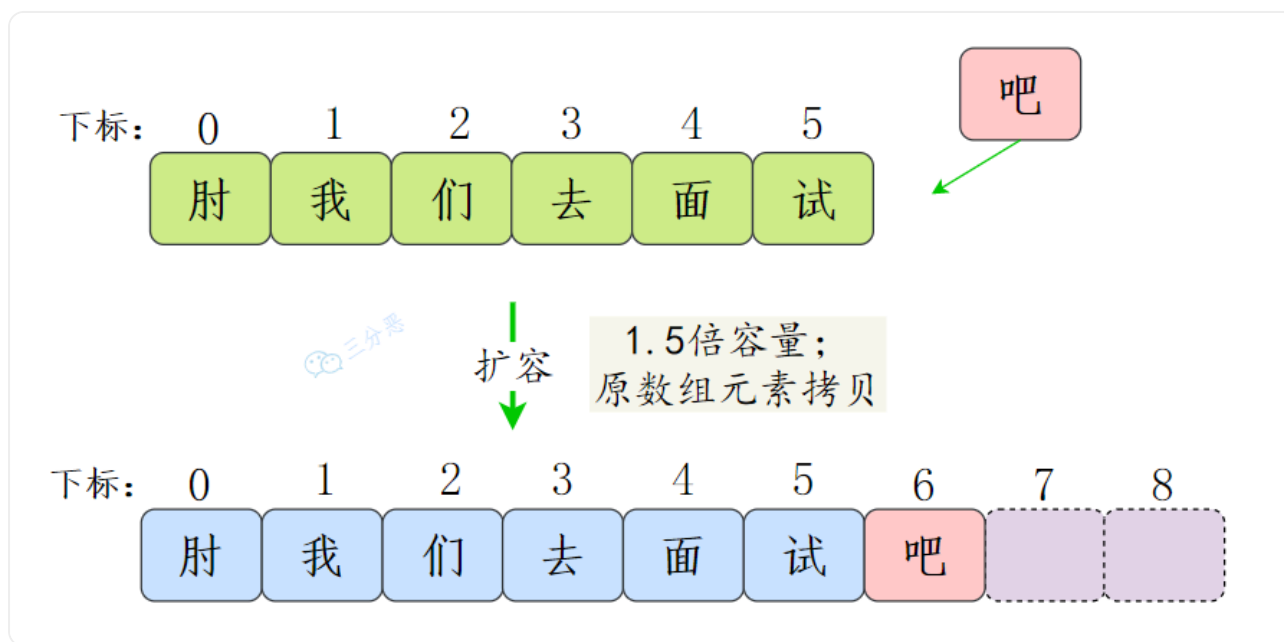
LinkedList 适用于：

- 频繁插入和删除：在列表中间频繁插入和删除元素的场景。
- 不需要快速随机访问：顺序访问多于随机访问的场景。
- 队列和栈：由于其双向链表的特性，LinkedList 可以高效地实现队列（FIFO）和栈（LIFO）。

1. [Java 面试指南（付费）](#) 收录的京东同学 10 后端实习一面的原题：ArrayList 和 LinkedList 的时间复杂度
2. [Java 面试指南（付费）](#) 收录的小米暑期实习同学 E 一面面试原题：你了解哪些集合？
3. [Java 面试指南（付费）](#) 收录的小米面经同学 F 面试原题：ArrayList和LinkedList的区别和使用场景

3.ArrayList 的扩容机制了解吗？

ArrayList 确切地说，应该叫做动态数组，因为它的底层是通过数组来实现的，当往 ArrayList 中添加元素时，会先检查是否需要扩容，如果当前容量+1 超过数组长度，就会进行扩容。



三分恶面渣逆袭：ArrayList扩容

扩容后的新数组长度是原来的 1.5 倍，然后再把原数组的值拷贝到新数组中。

```
private void grow(int minCapacity) {  
    // overflow-conscious code  
    int oldCapacity = elementData.length;  
    int newCapacity = oldCapacity + (oldCapacity >> 1);  
    if (newCapacity - minCapacity < 0)  
        newCapacity = minCapacity;  
    if (newCapacity - MAX_ARRAY_SIZE > 0)  
        newCapacity = hugeCapacity(minCapacity);  
    // minCapacity is usually close to size, so this is a win:  
    elementData = Arrays.copyOf(elementData, newCapacity);  
}
```

java

1. [Java 面试指南（付费）](#) 收录的联想面经同学 7 面试原题：Java 集合类介绍，挑一个讲原理。

4.ArrayList 怎么序列化的知道吗？ 为什么用 transient 修饰数组？

ArrayList 的序列化不太一样，它使用 transient 修饰存储元素的 `elementData` 的数组，`transient` 关键字的作用是让被修饰的成员属性不被序列化。

为什么最 ArrayList 不直接序列化元素数组呢？

出于效率的考虑，数组可能长度 100，但实际只用了 50，剩下的 50 不用其实不用序列化，这样可以提高序列化和反序列化的效率，还可以节省内存空间。

那 ArrayList 怎么序列化呢？

ArrayList 通过两个方法 `readObject`、`writeObject` 自定义序列化和反序列化策略，实际直接使用两个流 `ObjectOutputStream` 和 `ObjectInputStream` 来进行序列化和反序列化。

```
/**
 * 自定义序列化
 */
private void writeObject(java.io.ObjectOutputStream s)
    throws java.io.IOException{
    // fail-fast, 后续判断是否有并发处理
    int expectedModCount = modCount;
    // 序列化没有标记为 static、transient 的字段，包括 size 等。
    s.defaultWriteObject();

    s.writeInt(size);

    // 序列化数组的前size个元素
    for (int i=0; i<size; i++) {
        s.writeObject(elementData[i]);
    }

    if (modCount != expectedModCount) {
        throw new ConcurrentModificationException();
    }
}

/**
 * 自定义反序列化
 */
private void readObject(java.io.ObjectInputStream s)
    throws java.io.IOException, ClassNotFoundException {
    elementData = EMPTY_ELEMENTDATA;

    //反序列化没有标记为 static、transient 的字段，包括 size 等
    s.defaultReadObject();

    s.readInt(); // ignored

    if (size > 0) {
        // 数组扩容
        int capacity = calculateCapacity(elementData, size);
        SharedSecrets.getJavaOISAccess().checkArray(s, Object[].class, capacity);
        ensureCapacityInternal(size);

        Object[] a = elementData;
        // 反序列化元素并填充到数组中
        for (int i=0; i<size; i++) {
            a[i] = s.readObject();
        }
    }
}
```

ArrayList自定义序列化

5.快速失败(fail-fast)和安全失败(fail-safe)了解吗？

快速失败（fail-fast）：快速失败是 Java 集合的一种错误检测机制

- 在用迭代器遍历一个集合对象时，如果线程 A 遍历过程中，线程 B 对集合对象的内容进行了修改（增加、删除、修改），则会抛出 Concurrent Modification Exception。
- 原理：迭代器在遍历时直接访问集合中的内容，并且在遍历过程中使用一个 modCount 变量。集合在被遍历期间如果内容发生变化，就会改变 modCount 的值。每当迭代器使用 hasNext()/next() 遍历下一个元素之前，都会检测 modCount 变量是否为 expectedmodCount 值，是的话就返回遍历；否则抛出异常，终止遍历。
- 注意：这里异常的抛出条件是检测到 modCount != expectedmodCount 这个条件。如果集合发生变化时修改 modCount 值刚好又设置为了 expectedmodCount 值，则异常不会抛出。因此，不能依赖于这个异常是否抛出而进行并发操作的编程，这个异常只建议用于检测并发修改的 bug。
- 场景：java.util 包下的集合类都是快速失败的，不能在多线程下发生并发修改（迭代过程中被修改），比如 ArrayList 类。

安全失败（fail-safe）

- 采用安全失败机制的集合容器，在遍历时不是直接在集合内容上访问的，而是先复制原有集合内容，在拷贝的集合上进行遍历。
- 原理：由于迭代时是对原集合的拷贝进行遍历，所以在遍历过程中对原集合所作的修改并不能被迭代器检测到，所以不会触发 Concurrent Modification Exception。
- 缺点：基于拷贝内容的优点是避免了 Concurrent Modification Exception，但同样地，迭代器并不能访问到修改后的内容，即：迭代器遍历的是开始遍历那一刻拿到的集合拷贝，在遍历期间原集合发生的修改迭代器是不知道的。
- 场景：java.util.concurrent 包下的容器都是安全失败，可以在多线程下并发使用，并发修改，比如 CopyOnWriteArrayList 类。

6.有哪几种实现 ArrayList 线程安全的方法？

可以使用 Collections.synchronizedList() 方法，它将返回一个线程安全的 List。

```
SynchronizedList list = Collections.synchronizedList(new ArrayList()); java
```

内部是通过 [synchronized 关键字](#) 加锁来实现的。

也可以直接使用 [CopyOnWriteArrayList](#)，它是线程安全的，遵循写时复制的原则，每当对列表进行修改（例如添加、删除或更改元素）时，都会创建列表的一个新副本，这个新副本会替换旧的列表，而对旧列表的所有读取操作仍然可以继续。

```
CopyOnWriteArrayList list = new CopyOnWriteArrayList();
```

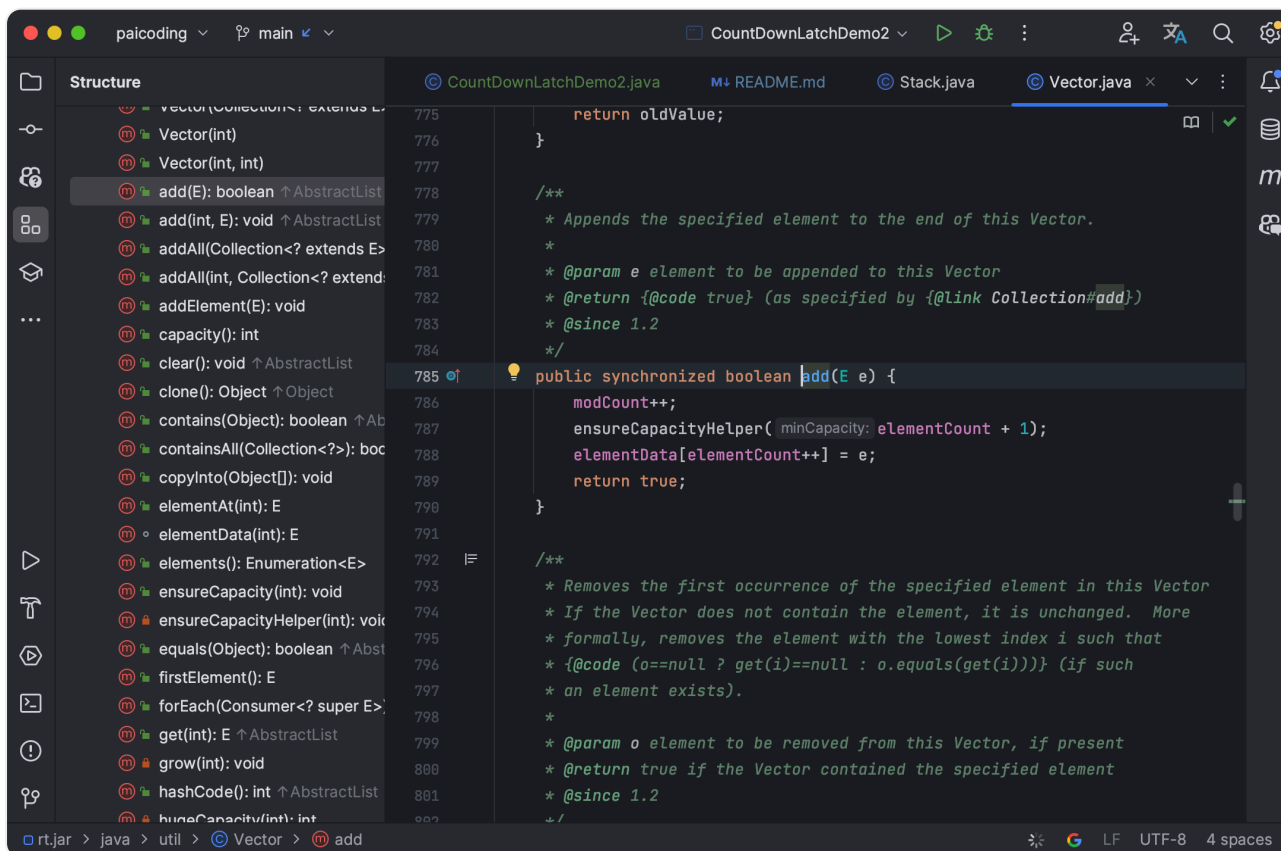
java

通俗的讲，CopyOnWrite 就是当我们往一个容器添加元素的时候，不直接往容器中添加，而是先复制出一个新的容器，然后在新容器里添加元素，添加完之后，再将原容器的引用指向新的容器。多个线程在读的时候，不需要加锁，因为当前容器不会添加任何元素。这样就实现了线程安全。

ArrayList 和 Vector 的区别？

Vector 属于 JDK 1.0 时期的遗留类，已不推荐使用，仍然保留着是因为 Java 希望向后兼容。

ArrayList 是在 JDK 1.2 时引入的，用于替代 Vector 作为主要的非同步动态数组实现。因为 Vector 所有的方法都使用 synchronized 关键字进行了同步，单线程环境下效率较低。



二哥的 Java 进阶之路：Vector源码

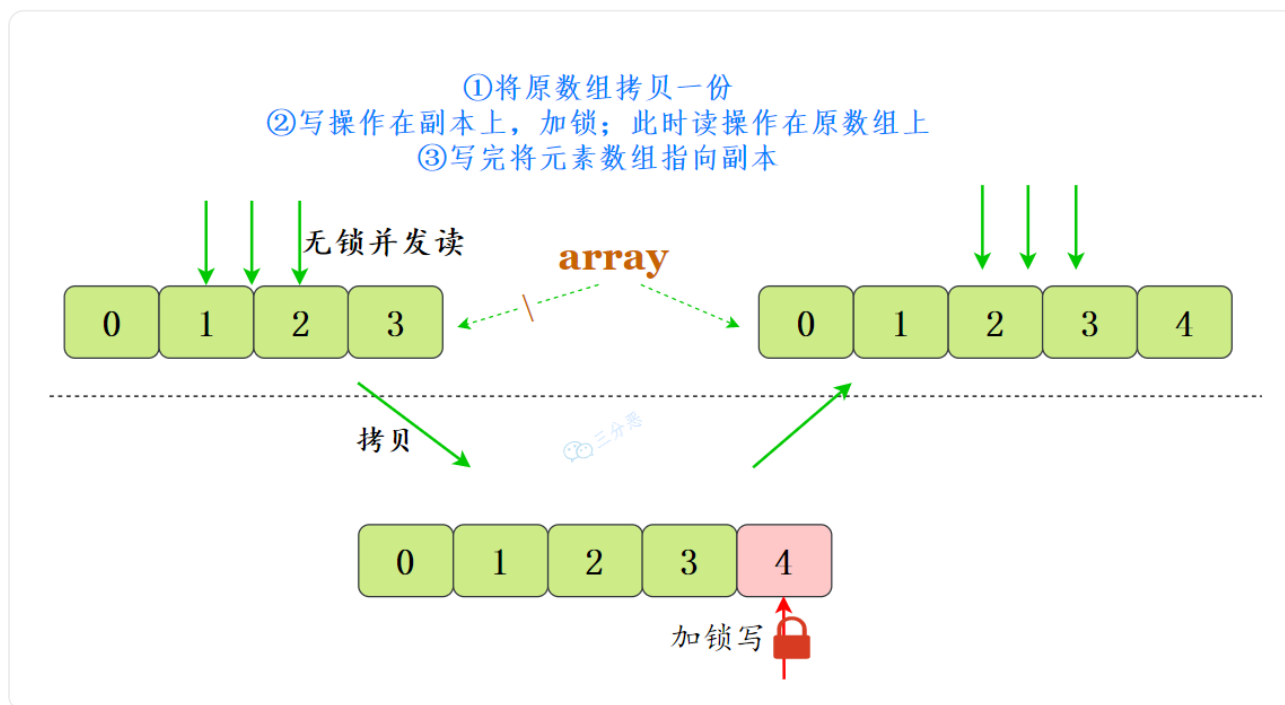
1. [Java 面试指南（付费）](#) 收录的招商银行面经同学 6 招银网络科技面试原题：线程不安全的集合变成线程安全的方法？
2. [Java 面试指南（付费）](#) 收录的 比亚迪面经同学2面试原题：ArrayList 和 vector 的区别

7.CopyOnWriteArrayList 了解多少？

CopyOnWriteArrayList 就是线程安全版本的 ArrayList。

它的名字叫 CopyOnWrite ——写时复制，已经明示了它的原理。

CopyOnWriteArrayList 采用了一种读写分离的并发策略。CopyOnWriteArrayList 容器允许并发读，读操作是无锁的，性能较高。至于写操作，比如向容器中添加一个元素，则首先将当前容器复制一份，然后在新副本上执行写操作，结束之后再原容器的引用指向新容器。



CopyOnWriteArrayList原理

GitHub 上标星 10000+ 的开源知识库《[二哥的 Java 进阶之路](#)》第一版 PDF 终于来了！包括 Java 基础语法、数组&字符串、OOP、集合框架、Java IO、异常处理、Java 新特性、网络编程、NIO、并发编程、JVM 等等，共计 32 万余字，500+张手绘图，可以说是通俗易懂、风趣幽默……详情戳：[太赞了，GitHub 上标星 10000+ 的 Java 教程](#)

微信搜 沉默王二 或扫描下方二维码关注二哥的原创公众号沉默王二，回复 222 即可免费领取。



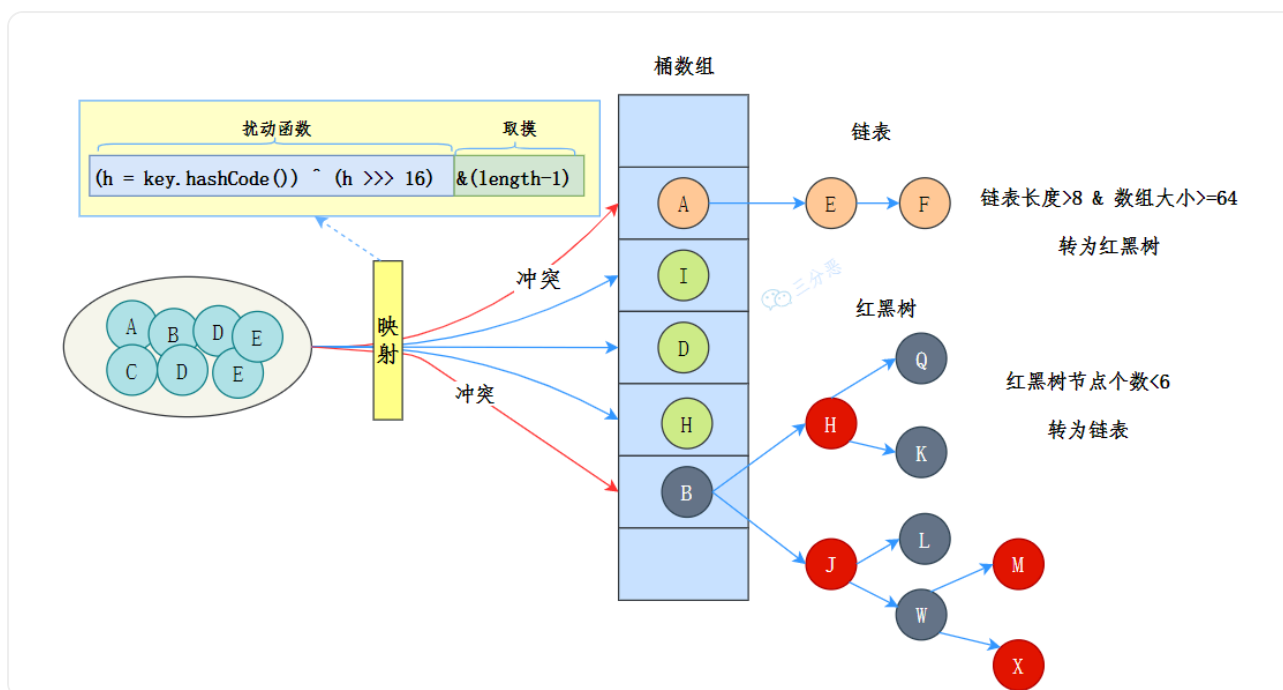
Map

Map 中，毫无疑问，最重要的就是 HashMap，面试基本被盘出包浆了，各种问法，一定要好好准备。

8.能说一下 HashMap 的底层数据结构吗？

推荐阅读：[二哥的 Java 进阶之路：详解 HashMap](#)

JDK 8 中 HashMap 的数据结构是 数组 + 链表 + 红黑树 。



三分恶面渣逆袭：JDK 8 HashMap 数据结构示意图

HashMap 的核心是一个动态数组（`Node[] table`），用于存储键值对。这个数组的每个元素称为一个“桶”（Bucket），每个桶的索引是通过将键的哈希值进行哈希函数处理得到的。

当多个键经哈希处理后得到相同的索引时，会发生哈希冲突。HashMap 通过链表来解决哈希冲突——即将具有相同索引的键值对通过链表连接起来。

不过，链表过长时，查询效率会比较低，于是当链表的长度超过 8 时（且数组的长度大于 64），链表就会转换为红黑树。红黑树的查询效率是 $O(\log n)$ ，比链表的 $O(n)$ 要快。数组的查询效率是 $O(1)$ 。

当向 HashMap 中添加一个键值对时，会使用哈希函数计算键的哈希码，确定其在数组中的位置，哈希函数的目标是尽量减少哈希冲突，保证元素能够均匀地分布在数组的每个位置上。

```
static final int hash(Object key) {  
    int h;  
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);  
}
```

java

当向 HashMap 中添加元素时，如果该位置已有元素（发生哈希冲突），则新元素将被添加到链表的末尾或红黑树中。如果键已经存在，其对应的值将被新值覆盖。

当从 HashMap 中获取元素时，也会使用哈希函数计算键的位置，然后根据位置在数组、链表或者红黑树中查找元素。

HashMap 的初始容量是 16，随着元素的不断添加，HashMap 的容量（也就是数组大小）可能不足，于是就需要进行扩容，阈值是 $\text{capacity} * \text{loadFactor}$ ，capacity 为容量，loadFactor 为负载因子，默认为 0.75。

扩容后的数组大小是原来的 2 倍，然后把原来的元素重新计算哈希值，放到新的数组中。

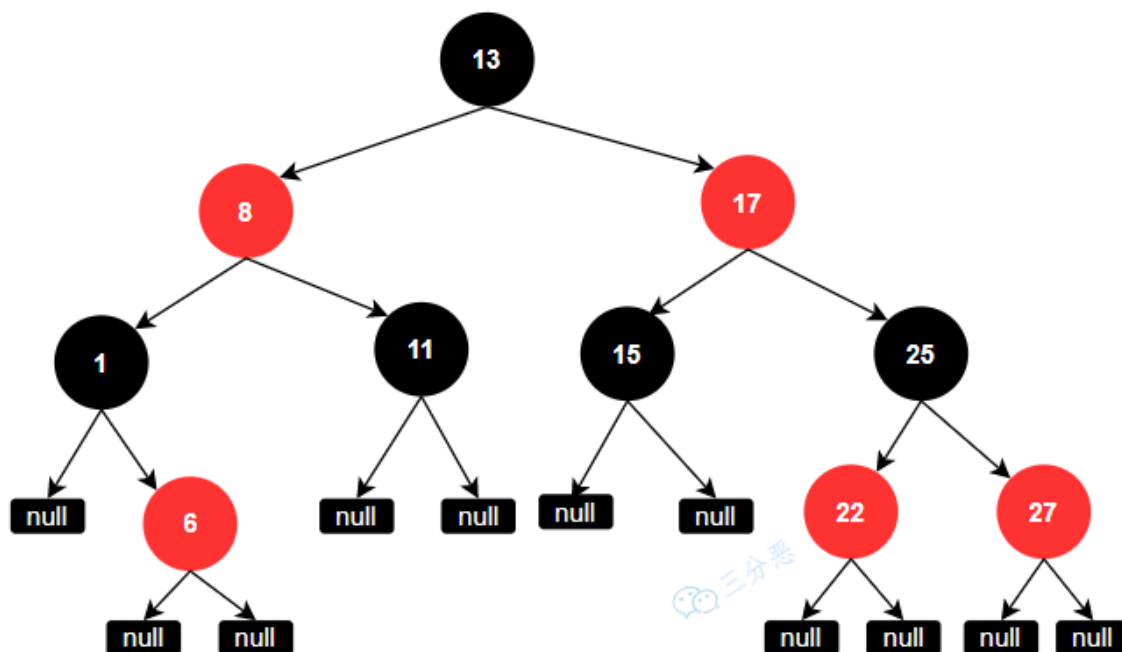
总的来说，HashMap 是一种通过哈希表实现的键值对集合，它通过将键哈希化成数组索引，并在冲突时使用链表或红黑树来存储元素，从而实现快速的查找、插入和删除操作。

1. [Java 面试指南（付费）](#) 收录的小米 25 届日常实习一面原题：讲一讲 HashMap 的原理
2. [Java 面试指南（付费）](#) 收录的华为一面原题：说下 Java 容器和 HashMap
3. [Java 面试指南（付费）](#) 收录的华为一面原题：说下 Redis 和 HashMap 的区别
4. [Java 面试指南（付费）](#) 收录的国企面试原题：说说 HashMap 的底层数据结构，链表和红黑树的转换，HashMap 的长度
5. [Java 面试指南（付费）](#) 收录的小米春招同学 K 一面面试原题：说一下 HashMap 数据库结构 和一些重要参数
6. [Java 面试指南（付费）](#) 收录的腾讯云智面经同学 16 一面面试原题：HashMap 的底层实现，它为什么是线程不安全的？
7. [Java 面试指南（付费）](#) 收录的快手面经同学 1 部门主站技术部面试原题：HashMap 的结构？
8. [Java 面试指南（付费）](#) 收录的百度面经同学 1 文心一言 25 实习 Java 后端面试原题：hashmap 的底层实现原理、put()方法实现流程、扩容机制？

9.你对红黑树了解多少？为什么不用二叉树/平衡树呢？

红黑树是一种自平衡的二叉查找树：

1. 每个节点要么是红色，要么是黑色；
2. 根节点永远是黑色；
3. 所有的叶子节点都是黑色的（下图中的 NULL 节点）；
4. 红色节点的子节点一定是黑色的；
5. 从任一节点到其每个叶子的所有简单路径都包含相同数目的黑色节点。



三分恶面渣逆袭：红黑树

为什么不用二叉树？

二叉树是最基本的树结构，每个节点最多有两个子节点，但是二叉树容易出现极端情况，比如插入的数据是有序的，那么二叉树就会退化成链表，查询效率就会变成 $O(n)$ 。

为什么不用平衡二叉树？

平衡二叉树比红黑树的要求更高，每个节点的左右子树的高度最多相差 1，这种高度的平衡保证了极佳的查找效率，但在进行插入和删除操作时，可能需要频繁地进行旋转来维持树的平衡，这在某些情况下可能导致更高的维护成本。

红黑树是一种折中的方案，它在保证了树平衡的同时，插入和删除操作的性能也得到了保证，查询效率是 $O(\log n)$ 。

1. [Java 面试指南（付费）](#) 收录的携程面经同学 1 Java 后端技术一面试原题：HashMap 为什么用红黑树，链表转数条件，红黑树插入删除规则

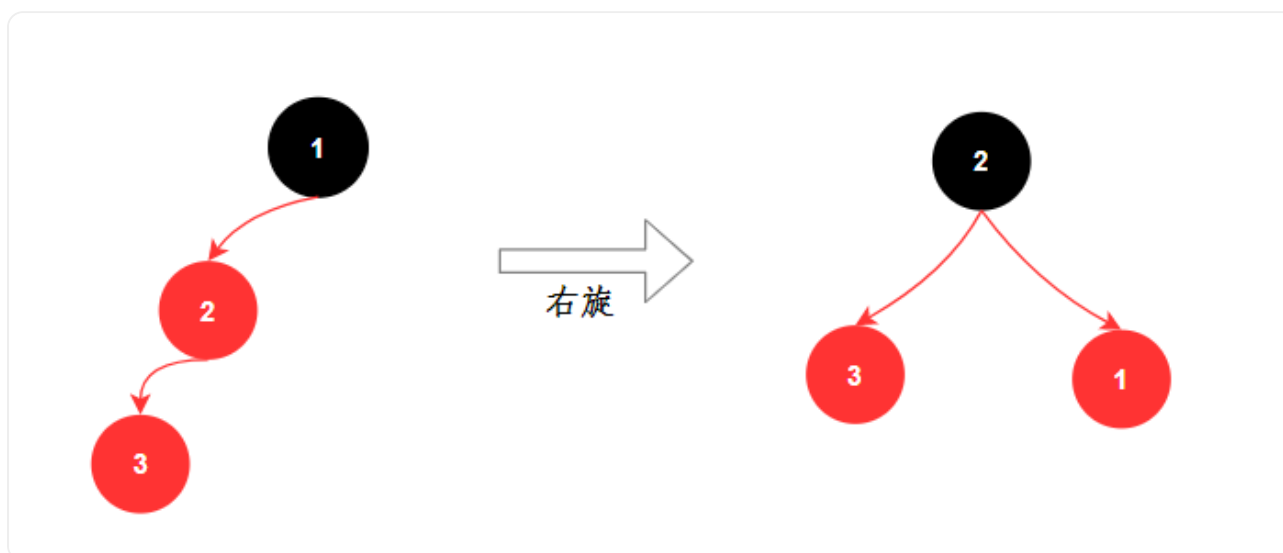
10.红黑树怎么保持平衡的？

红黑树有两种方式保持平衡： 旋转 和 染色 。

①、旋转：旋转分为两种，左旋和右旋

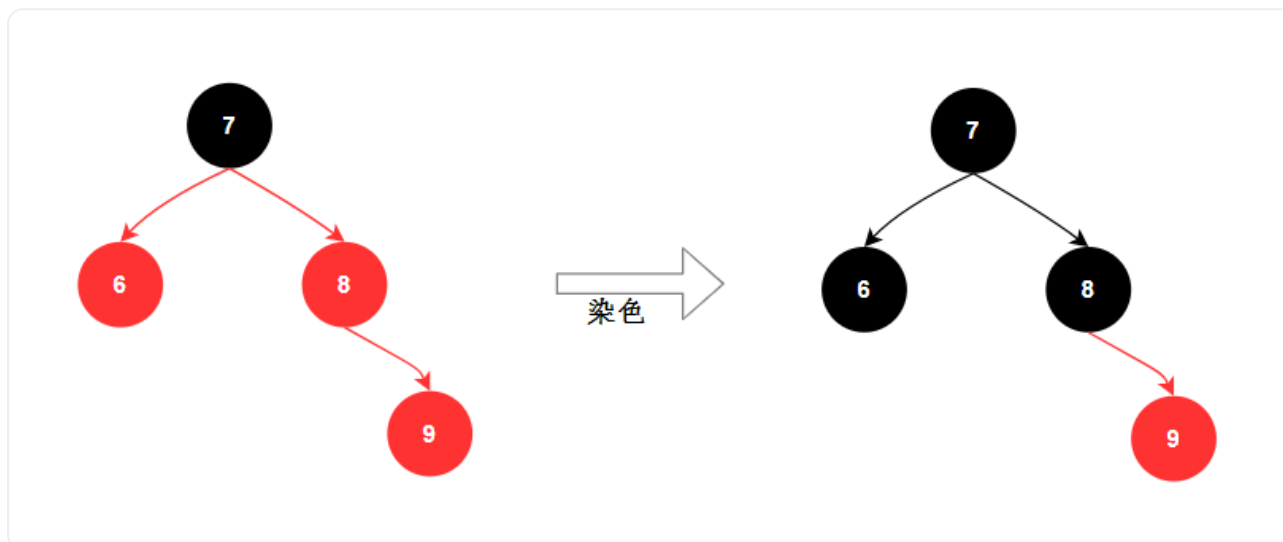


三分恶面渣逆袭：左旋



三分恶面渣逆袭：右旋

②、染色：

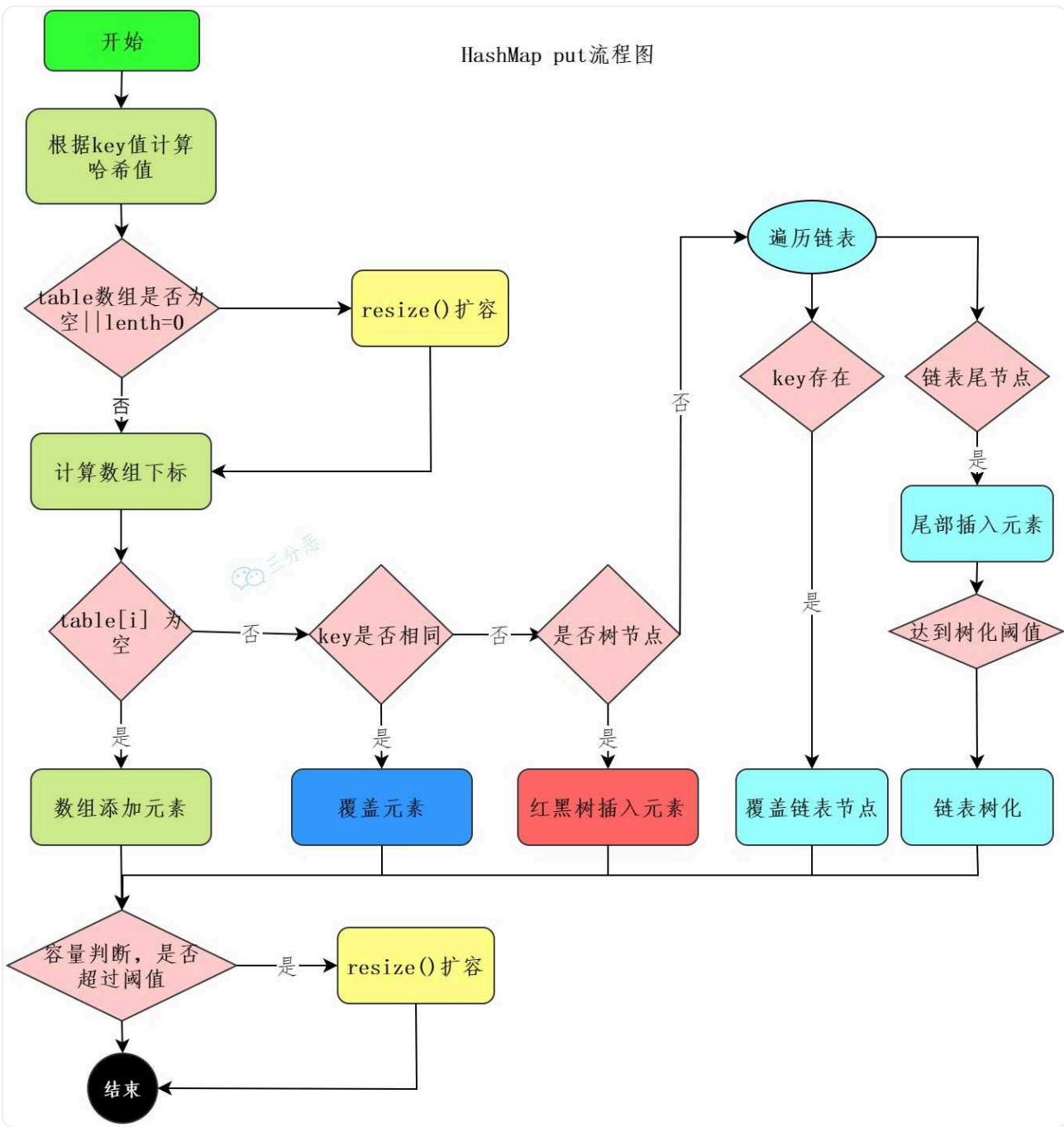


三分恶面渣逆袭：染色

1. [Java 面试指南（付费）](#) 收录的携程面经同学 1 Java 后端技术一面面试原题：
HashMap 为什么用红黑树，链表转数条件，红黑树插入删除规则

11.HashMap 的 put 流程知道吗？

直接看流程图。



三分恶面渣逆袭：HashMap插入数据流程图

第一步，通过 hash 方法计算 key 的哈希值。

```
static final int hash(Object key) {  
    int h;  
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >> 16);  
}
```

java

第二步，数组进行第一次扩容。

```
if ((tab = table) == null || (n = tab.length) == 0)  
    n = (tab = resize()).length;
```

java

第三步，根据哈希值计算 key 在数组中的下标，如果对应下标正好没有存放数据，则直接插入。

```
if ((p = tab[i = (n - 1) & hash]) == null)  
    tab[i] = newNode(hash, key, value, null);
```

java

如果对应下标已经有数据了，就需要判断是否为相同的 key，是则覆盖 value，否则需要判断是否为树节点，是则向树中插入节点，否则向链表中插入数据。

```
else {  
    Node<K,V> e; K k;  
    if (p.hash == hash &&  
        ((k = p.key) == key || (key != null && key.equals(k))))  
        e = p;  
    else if (p instanceof TreeNode)  
        e = ((TreeNode<K,V>)p).putTreeVal(this, tab, hash, key, value);  
    else {  
        for (int binCount = 0; ; ++binCount) {  
            if ((e = p.next) == null) {  
                p.next = newNode(hash, key, value, null);  
                if (binCount >= TREEIFY_THRESHOLD - 1) // -1 for 1st  
                    treeifyBin(tab, hash);  
                break;  
            }  
            if (e.hash == hash &&  
                ((k = e.key) == key || (key != null && key.equals(k))))
```

java

```
        break;
        p = e;
    }
}
```

注意，在链表中插入节点的时候，如果链表长度大于等于 8，则需要把链表转换为红黑树。

```
if (binCount >= TREEIFY_THRESHOLD - 1) // -1 for 1st
    treeifyBin(tab, hash);
```

java

所有元素处理完后，还需要判断是否超过阈值 threshold，超过则扩容。

```
if (++size > threshold)
    resize();
```

java

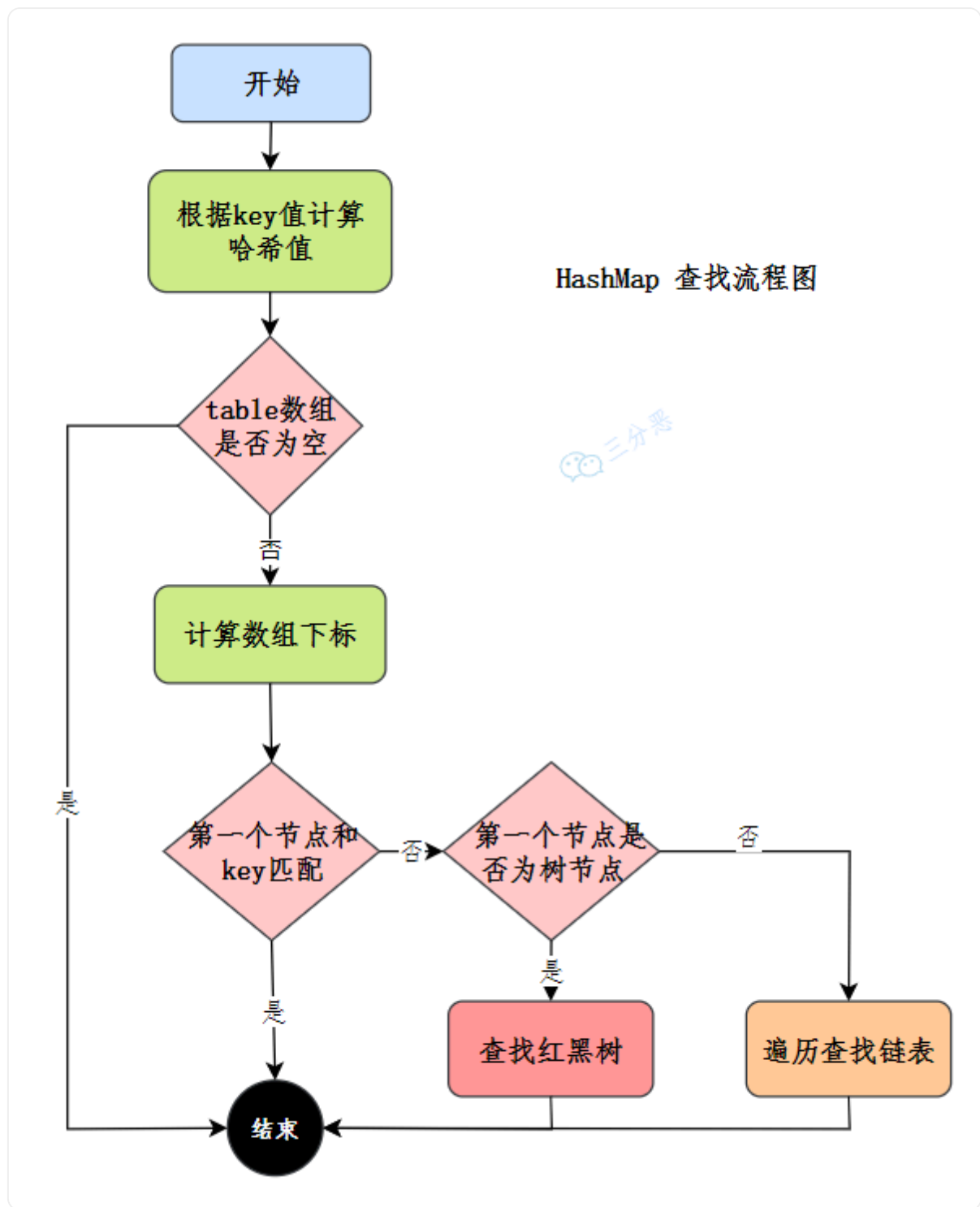
只重写 equals 没重写 hashCode，map put 的时候会发生什么？

如果只重写 equals 方法，没有重写 hashCode 方法，那么会导致 equals 相等的两个对象，hashCode 不相等，这样的话，这两个对象会被放到不同的桶中，这样就会导致 get 的时候，找不到对应的值。

1. [Java 面试指南（付费）](#) 收录的京东同学 10 后端实习一面的原题：hashCode 和 equals 方法只重写一个行不行，只重写 equals 没重写 hashCode，map put 的时候会发生什么
2. [Java 面试指南（付费）](#) 收录的快手面经同学 1 部门主站技术部面试原题：HashMap 的 put 过程
3. [Java 面试指南（付费）](#) 收录的百度面经同学 1 文心一言 25 实习 Java 后端面试原题：hashmap 的底层实现原理、put()方法实现流程、扩容机制？

12.HashMap 怎么查找元素的呢？

先看流程图：



HashMap查找流程图

HashMap 的查找就简单很多：

1. 使用扰动函数，获取新的哈希值
2. 计算数组下标，获取节点
3. 当前节点和 key 匹配，直接返回
4. 否则，当前节点是否为树节点，查找红黑树
5. 否则，遍历链表查找

13.HashMap 的 hash 函数是怎么设计的？

HashMap 的哈希函数是先拿到 key 的 hashCode，是一个 32 位的 int 类型的数值，然后让 hashCode 的高 16 位和低 16 位进行异或操作。

```
static final int hash(Object key) {  
    int h;  
    // key的hashCode和key的hashCode右移16位做异或运算  
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);  
}
```

java

这么设计是为了降低哈希碰撞的概率。

14.为什么 hash 函数能降哈希碰撞？

```
static final int hash(Object key) { Complexity is 6 It's time to do something...  
    int h;  
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);  
}
```

JDK 8

在 hash 函数中，先调用了 key 的 hashCode() 方法，这将会返回一个 int 类型的哈希值，比如说字符串的 hashCode。

```
public int hashCode() {  
    int h = hash;  
    if (h == 0 && value.length > 0) {  
        char val[] = value;  
  
        for (int i = 0; i < value.length; i++) {  
            h = 31 * h + val[i];  
        }  
        hash = h;  
    }  
    return h;  
}
```

java

int 的范围是 -2147483648~2147483647，加起来大概 40 亿上下的浮动。

只要 key 的 hashCode 方法设计的比较合理，一般是很难出现碰撞的。但问题是，不可能直接搞一个 40 亿长度的数组啊，那也太铺张浪费了。

我们一般会设置一个较小的数组长度，比如说 HashMap 的数组初始大小才 16，当发现容量不满足的时候再扩容，避免浪费。

那当数组长度比较小的时候，我们就需要设计一种比较巧妙的 hash 算法，来避免发生哈希冲突，尽可能地让元素均匀地分布在数组当中。

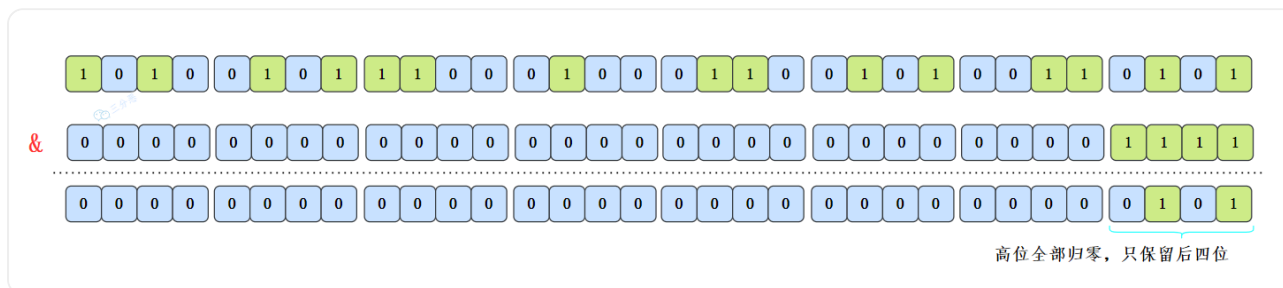
要达到这个目的，HashMap 在两方面下足了功夫，第一个就是数组的长度必须是 2 的整数次幂，这样可以保证 $\text{hash} \& (n-1)$ 的结果能均匀地分布在数组中。

其作用就相当于 $\text{hash} \% n$ ，n 为数组的长度，比如说数组长度是 16，hash 值为 20，那么 $20 \% 16 = 4$ ，也就是说 20 这个元素应该放在数组的第 4 个位置；hash 值为 23，那么 $23 \% 16 = 7$ ，也就是说 23 这个元素应该放在数组的第 7 个位置。

& 操作的结果就是哈希值的高位全部归零，只保留 n 个低位，用来做数组下标访问。

比如说 $\text{hash} \& (2^4 - 1)$ 的结果实际上是取 hash 的低 4 位，这四位能表示的取值范围刚好是 0000 到 1111，也就是 0 到 15，正好是数组长度为 16 的下标范围。

以初始长度 16 为例， $16-1=15$ 。2 进制表示是 0000 0000 0000 0000 0000 0000 0000 1111。和某个哈希值做 & 运算，结果就是截取了最低的四位。



三分恶面渣逆袭：哈希&运算

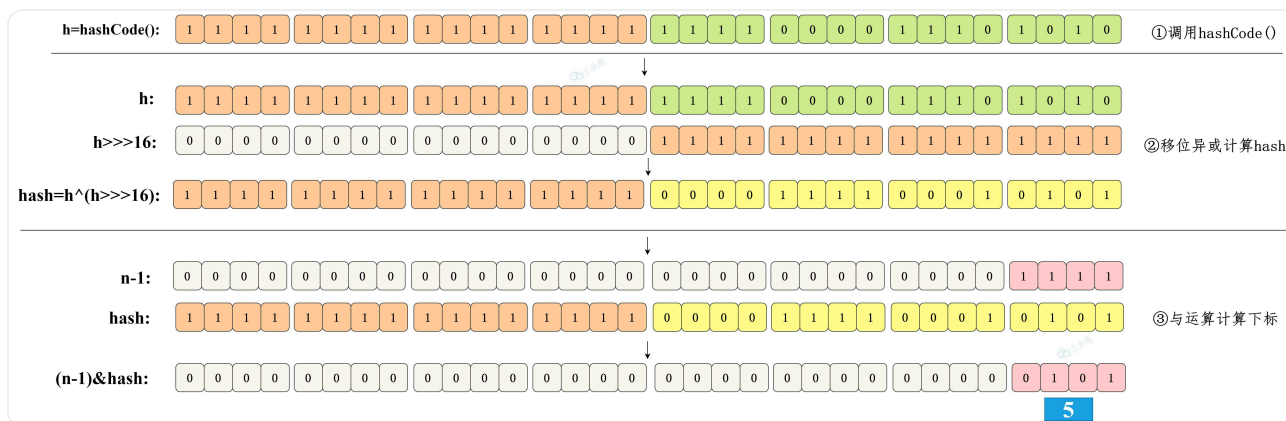
那问题又来了，那么大一个哈希值，也只取最后 4 位，不就等于哈希值的高位都丢弃了吗？

比如说 1111 1111 1111 1111 1111 1111 1111 1111，取最后 4 位，也就是 1111。

比如说 1110 1111 1111 1111 1111 1111 1111 1111，取最后 4 位，也是 1111。

不就发生哈希冲突了吗？

这时候 hash 函数 $(h = \text{key.hashCode()}) \wedge (h \ggg 16)$ 就派上用场了呀。



三分恶面渣逆袭：hash 函数示意图

将哈希值无符号右移 16 位，意味着原哈希值的高 16 位被移到了低 16 位的位置。这样，原始哈希值的高 16 位和低 16 位就可以参与到最终用于索引计算的低位中。

选择 16 位是因为它是 32 位整数的一半，这样处理既考虑了高位的信息，又没有完全忽视低位原本的信息，尝试达到一个平衡状态。

举个例子（数组长度为 16）。

- 第一个数：h1 = 0001 0010 0011 0100 0101 0110 0111 1000
- 第二个数：h2 = 0001 0010 0011 0101 0101 0110 0111 1000

如果没有 hash 函数，直接取低 4 位，那么 h1 和 h2 的低 4 位都是 1000，也就是两个数都会放在数组的第 8 个位置。

来看一下 hash 函数的处理过程。

①、对于第一个数 h1 的计算：

```
原始：0001 0010 0011 0100 0101 0110 0111 1000
右移：0000 0000 0000 0000 0001 0010 0011 0100
异或：-----
结果：0001 0010 0011 0100 0100 0100 0100 1100
```

②、对于第二个数 h2 的计算：

```
原始：0001 0010 0011 0101 0101 0110 0111 1000
右移：0000 0000 0000 0000 0001 0010 0011 0101
异或：-----
结果：0001 0010 0011 0101 0100 0100 0100 1101
```

通过上述计算，我们可以看到 h1 和 h2 经过 `h ^ (h >> 16)` 操作后得到了不同的结果。

现在，考虑数组长度为 16 时（需要最低 4 位来确定索引）：

- 对于 h1 的最低 4 位是 1100（十进制中为 12）
- 对于 h2 的最低 4 位是 1101（十进制中为 13）

这样，h1 和 h2 就会被分别放在数组的第 12 个位置和第 13 个位置上，避免了哈希冲突。

1. [Java 面试指南（付费）](#) 收录的支付宝面经同学 2 春招技术一面面试原题：为什么要用高低做异或运算？为什么非得高低 16 位异或？

15.为什么 HashMap 的容量是 2 的倍数呢？

HashMap 的容量是 2 的倍数，或者说是 2 的整数次幂，是为了快速定位元素的下标：

HashMap 在定位元素位置时，先通过 $\text{hash}(\text{key}) = (\text{h} = \text{key}.\text{hashCode}()) \wedge (\text{h} \ggg 16)$ 计算出哈希值，再通过 $\text{hash} \& (\text{n}-1)$ 来定位元素位置的，n 为数组的大小，也就是 HashMap 的容量。

因为（数组长度-1）正好相当于一个“低位掩码”——这个掩码的低位最好全是 1，这样 & 操作才有意义，否则结果就肯定是 0。

a&b 操作的结果是：a、b 中对应位同时为 1，则对应结果位为 1，否则为 0。例如 $5 \& 3 = 1$ ，5 的二进制是 0101，3 的二进制是 0011， $5 \& 3 = 0001 = 1$ 。

2 的整次幂（或者叫 2 的整数倍）刚好是偶数，偶数-1 是奇数，奇数的二进制最后一位是 1，保证了 $\text{hash} \& (\text{length}-1)$ 的最后一位可能为 0，也可能为 1（取决于 hash 的值），即 & 运算后的结果可能为偶数，也可能为奇数，这样便可以保证哈希值的均匀分布。

换句话说，& 操作的结果就是将哈希值的高位全部归零，只保留低位值。

假设某哈希值的二进制为 10100101 11000100 00100101，用它来做 & 运算，我们来看一下结果。

我们知道，HashMap 的初始长度为 16， $16-1=15$ ，二进制是 00000000 00000000 00001111（高位用 0 来补齐）：

```
10100101 11000100 00100101
& 00000000 00000000 00001111
-----
00000000 00000000 00000101
```

因为 15 的高位全部是 0，所以 & 运算后的高位结果肯定也是 0，只剩下 4 个低位 0101，也就是十进制的 5。

这样，哈希值为 10100101 11000100 00100101 的键就会放在数组的第 5 个位置上。

hashCode 对数组长度取模定位数组下标，这块有没有优化策略？

我先说一下取模运算。

取模运算（Modulo Operation）和取余运算（Remainder Operation）从严格意义上来讲，是两种不同的运算方式，它们在计算机中的实现也不同。

在 Java 中，通常使用 % 运算符来表示取余，用 Math.floorMod() 来表示取模。

- 当操作数都是正数的话，取模运算和取余运算的结果是一样的。
- 只有当操作数出现负数的情况，结果才会有所不同。
- **取模运算的商向负无穷靠近；取余运算的商向 0 靠近。**这是导致它们两个在处理有负数情况下，结果不同的根本原因。
- 当数组的长度是 2 的 n 次方，或者 n 次幂，或者 n 的整数倍时，取模运算/取余运算可以用位运算来代替，效率更高，毕竟计算机本身只认二进制嘛。

比如说，7 对 3 取余，和 7 对 3 取模，结果都是 1。因为两者都是基于除法运算的， $7 / 3$ 的商是 2，余数是 1。

于是，我们会在很多地方看到，**取余就是取模，取模就是取余。**。

对于 HashMap 来说，它需要通过 `hash % table.length` 来确定元素在数组中的位置，这种做法可以在很大程度上让元素均匀的分布在数组中。

比如说，数组长度是 3，hash 是 7，那么 $7 \% 3$ 的结果就是 1，也就是此时可以把元素放在下标为 1 的位置。

当 hash 是 8， $8 \% 3$ 的结果就是 2，也就是可以把元素放在下标为 2 的位置。

当 hash 是 9， $9 \% 3$ 的结果就是 0，也就是可以把元素放在下标为 0 的位置上。

是不是很奇妙，数组的大小为 3，刚好 3 个位置都利用上了。

那为什么 HashMap 在计算下标的时候，并没有直接使用取余运算（或者取模运算），而是直接使用位与运算 & 呢？

这其实就是 HashMap 的一个优化策略。

因为当数组的长度是 2 的 n 次方时， $hash \& (length - 1) = hash \% length$ 。

比如说 $9 \% 4 = 1$ ，9 的二进制是 1001， $4 - 1 = 3$ ，3 的二进制是 0011， $9 \& 3 = 1001 \& 0011 = 0001 = 1$ 。

再比如说 $10 \% 4 = 2$ ，10 的二进制是 1010， $4 - 1 = 3$ ，3 的二进制是 0011， $10 \& 3 = 1010 \& 0011 = 0010 = 2$ 。

当数组的长度不是 2 的 n 次方时， $\text{hash} \% \text{length}$ 和 $\text{hash} \& (\text{length} - 1)$ 的结果就不一致了。

比如说 $7 \% 3 = 1$ ，7 的二进制是 0111， $3 - 1 = 2$ ，2 的二进制是 0010， $7 \& 2 = 0111 \& 0010 = 0010 = 2$ 。

因为从二进制角度来看， $\text{hash} / \text{length} = \text{hash} / 2^n = \text{hash} \gg n$ ，即把 hash 右移 n 位，此时得到了 $\text{hash} / 2^n$ 的商。

而被移调的部分，则是 $\text{hash} \% 2^n$ ，也就是余数。

2^n 的二进制形式为 1，后面跟着 n 个 0，那 $2^n - 1$ 的二进制则是 n 个 1。例如 $8 = 2^3$ ，二进制是 1000， $7 = 2^3 - 1$ ，二进制为 0111。

$\text{hash} \% \text{length}$ 的操作是求 hash 除以 2^n 的余数。在二进制中，这个操作的结果就是 hash 的二进制表示中最低 n 位的值。

因为在 2^n 取模的操作中，高于 2^n 表示位的所有数值对结果没有贡献，只有低于这个阈值的部分才决定余数。

比如说 26 的二进制是 11010，要计算 $26 \% 8$ ，8 是 2^3 ，所以我们关注的是 26 的二进制表示中最低 3 位：11010 的最低 3 位是 010。

010 对应于十进制中的 2， $26 \% 8$ 的结果是 2。

当执行 $\text{hash} \& (\text{length} - 1)$ 时，实际上是保留 hash 二进制表示的最低 n 位，其他高位都被清零。

& 与运算：两个操作数中位都为 1，结果才为 1，否则结果为 0。

举个例子，hash 为 14， n 为 3，也就是数组长度为 2^3 ，也就是 8。

```
1110 (hash = 14)
& 0111 (length - 1 = 7)
----
0110 (结果 = 6)
```

保留 14 的最低 3 位，高位被清零。

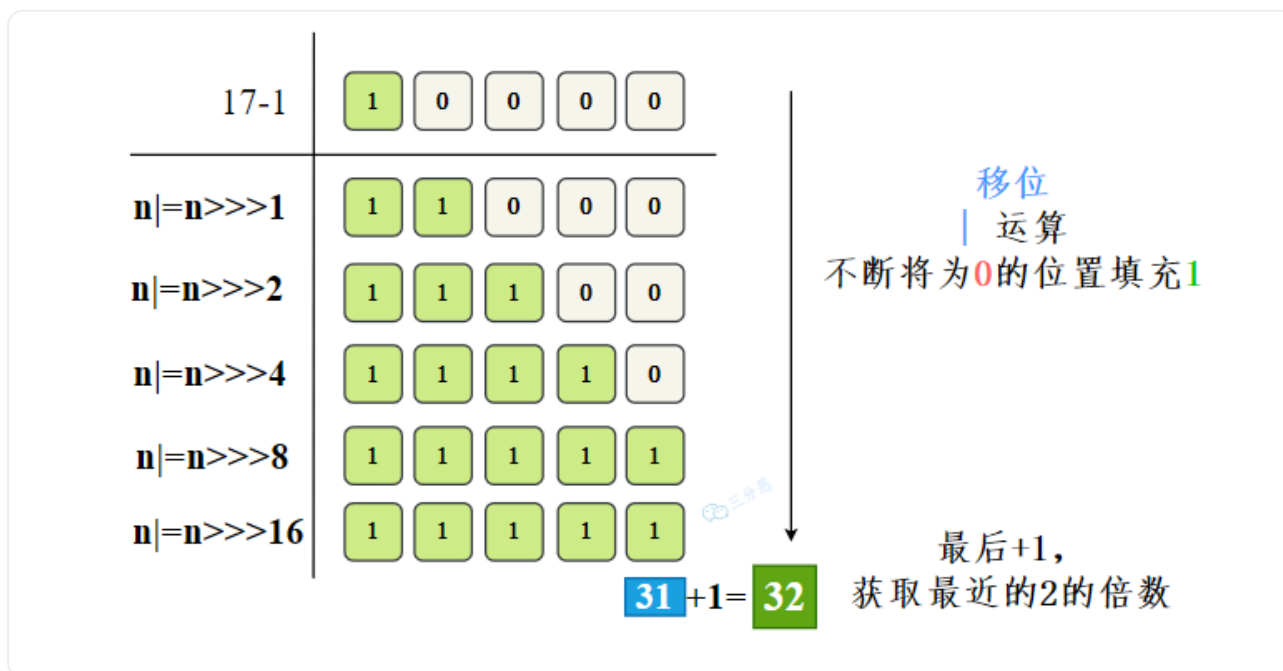
从此，两个运算 $\text{hash} \% \text{length}$ 和 $\text{hash} \& (\text{length} - 1)$ 有了完美的闭环。在计算机中，位运算的速度要远高于取余运算，因为计算机本质上就是二进制嘛。

1. [Java 面试指南（付费）](#) 收录的小米春招同学 K 一面面试原题：为什么是 2 次幂 到什么时候开始扩容 扩容机制流程
2. [Java 面试指南（付费）](#) 收录的支付宝面经同学 2 春招技术一面面试原题：
`hashCode` 对数组长度取模定位数组下标，这块有没有优化策略？

16.如果初始化 HashMap，传一个 17 容量，它会怎么处理？

HashMap 会将这个值转换为大于或等于 17 的最小的 2 的幂。这是因为 HashMap 的设计是基于哈希表的，而哈希表的大小最好是 2 的幂，这样可以优化哈希值的计算，并减少哈希冲突。

所以，如果你传入 17 作为初始容量，HashMap 实际上会被初始化为大小为 32 的哈希表。



三分恶面渣逆袭：容量计算

在 HashMap 的初始化构造方法中，有这样一段代码：

```
public HashMap(int initialCapacity, float loadFactor) {  
    ...  
    this.loadFactor = loadFactor;  
    this.threshold = tableSizeFor(initialCapacity);  
}
```

阈值 threshold 会通过方法 `tableSizeFor()` 进行计算。

```
static final int tableSizeFor(int cap) {  
    int n = cap - 1;  
    n |= n >>> 1;  
    n |= n >>> 2;  
    n |= n >>> 4;  
    n |= n >>> 8;  
    n |= n >>> 16;  
    return (n < 0) ? 1 : (n >= MAXIMUM_CAPACITY) ? MAXIMUM_CAPACITY : n +  
1;  
}
```

java

①、`int n = cap - 1;` 将传入的容量减 1，用于确保如果传入的容量已经是 2 的幂次方，计算结果不会超过这个数。

②、接下来通过不断右移（`>>>`）并与自身进行或运算（`|=`），将 `n` 的二进制表示中的所有低位设置为 1。

- `n |= n >>> 1;` 把 `n` 的二进制表示中最高位的 1 之后的一个 0 变成 1。
- `n |= n >>> 2;` 接着把后两位中的 0 都变成 1。
- 依此类推，直到 `n |= n >>> 16;`，此时 `n` 的二进制表示中，从最高位的 1 开始到最低位，都变成了 1。

③、如果 `n` 小于 0，说明 `cap` 是负数，直接返回 1（理论上哈希表的大小不应该是负数或 0）。

如果 `n` 大于或等于 `MAXIMUM_CAPACITY`（通常是 2^{30} ），则返回 `MAXIMUM_CAPACITY`。

否则，返回 `n + 1`，这是因为 `n` 的所有低位都是 1，所以 `n + 1` 就是大于 `cap` 的最小的 2 的幂次方。

初始化 HashMap 的时候需要传入容量值吗？

在创建 `HashMap` 时可以指定初始容量值。这个容量是指 `Map` 内部用于存储数据的数组大小。

如果预先知道 `Map` 将存储大量键值对，提前指定一个足够大的初始容量可以减少因扩容导致的重哈希（`rehashing`）操作，从而提高性能。

因为每次扩容时，`HashMap` 需要新分配一个更大的数组并重新将现有的元素插入到这个新数组中，这个过程相对耗时，尤其是当 `Map` 中已有大量数据时。

当然了，过大的初始容量会浪费内存，特别是当实际存储的元素远少于初始容量时。如果不指定初始容量，HashMap 将使用默认的初始容量 16。

1. [Java 面试指南（付费）](#) 收录的奇安信面经同学 1 Java 技术一面面试原题：map 集合在使用时一般都需要写容量值？为什么要写？扩容机制？

17.你还知道哪些哈希函数的构造方法呢？

HashMap 里哈希构造函数的方法叫：

- **除留取余法：** $H(key) = key \% p (p \leq N)$ ，关键字除以一个不大于哈希表长度的正整数 p ，所得余数为地址，当然 HashMap 里进行了优化改造，效率更高，散列也更均衡。

除此之外，还有这几种常见的哈希函数构造方法：

- **直接定址法**

直接根据 key 来映射到对应的数组位置，例如 1232 放到下标 1232 的位置。

- **数字分析法**

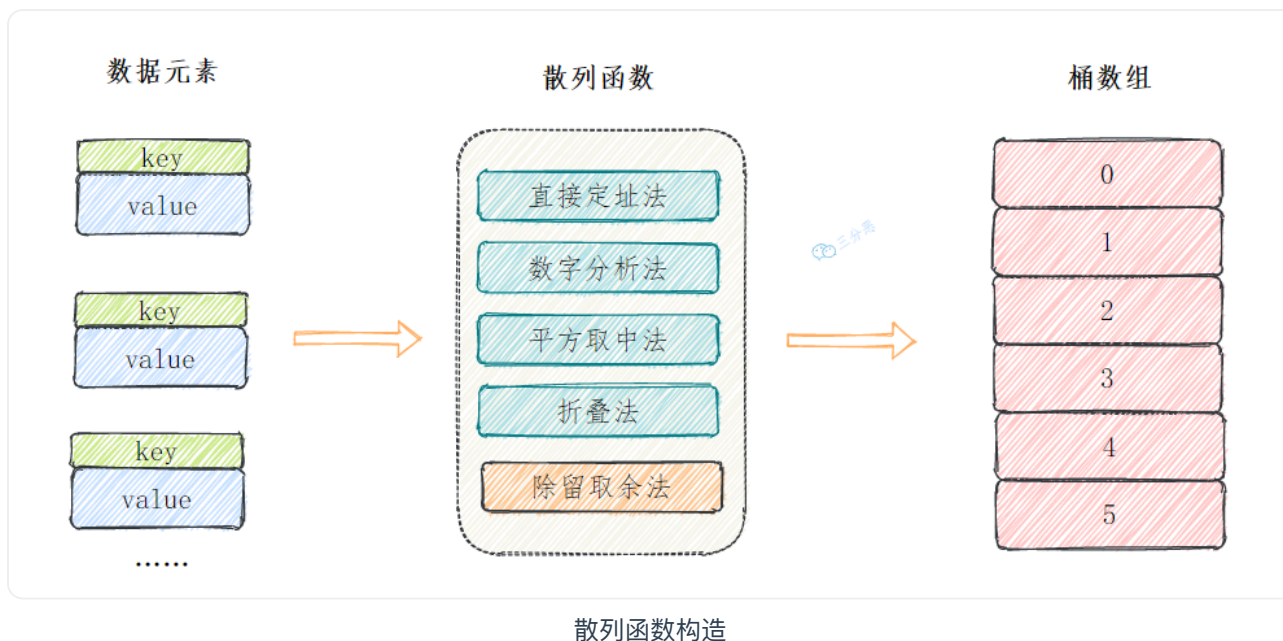
取 key 的某些数字（例如十位和百位）作为映射的位置

- **平方取中法**

取 key 平方的中间几位作为映射的位置

- **折叠法**

将 key 分割成位数相同的几段，然后把它们的叠加和作为映射的位置



18.解决哈希冲突有哪些方法呢？

解决哈希冲突的方法我知道的有 3 种：

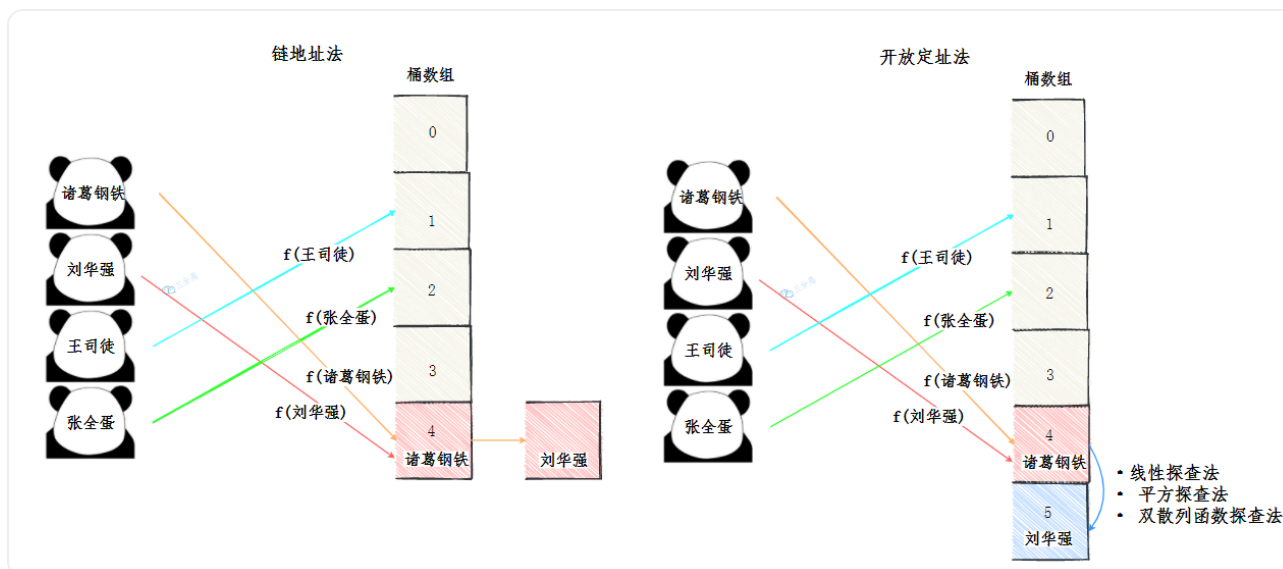
①、再哈希法

准备两套哈希算法，当发生哈希冲突的时候，使用另外一种哈希算法，直到找到空槽为止。对哈希算法的设计要求比较高。

②、开放地址法

遇到哈希冲突的时候，就去寻找下一个空的槽。有 3 种方法：

- 线性探测：从冲突的位置开始，依次往后找，直到找到空槽。
- 二次探测：从冲突的位置 x 开始，第一次增加 1^2 个位置，第二次增加 2^2 ，直到找到空槽。
- 双重哈希：和再哈希法类似，准备多个哈希函数，发生冲突的时候，使用另外一个哈希函数。



三分恶面渣逆袭：拉链法 VS 开放地址法

③、拉链法

也就是所谓的链地址法，当发生哈希冲突的时候，使用链表将冲突的元素串起来。
HashMap 采用的正是拉链法。

怎么判断 key 相等呢？

HashMap 判断两个 key 是否相等，依赖于 key 的 `equals()` 方法和 `hashCode()` 方法，以及 `==` 运算符。

```
if (e.hash == hash &&
    ((k = e.key) == key || (key != null && key.equals(k))))
```

java

- ①、**hashCode()**：首先，使用 key 的 `hashCode()` 方法计算 key 的哈希码。由于不同的 key 可能有相同的哈希码，`hashCode()` 只是第一步筛选。
- ②、**equals()**：当两个 key 的哈希码相同时，HashMap 还会调用 key 的 `equals()` 方法进行精确比较。只有当 `equals()` 方法返回 `true` 时，两个 key 才被认为是完全相同的。
- ③、**==**：当然了，如果两个 key 的引用指向同一个对象，那么它们的 `hashCode()` 和 `equals()` 方法都会返回 `true`，所以在 `equals` 判断之前会优先使用 `==` 运算符判断一次。

1. [Java 面试指南（付费）](#) 收录的支付宝面经同学 2 春招技术一面面试原题：
HashMap 怎么解决冲突？怎么判断 key 相等？

19.为什么 HashMap 链表转红黑树的阈值为 8 呢？

树化发生在 table 数组的长度大于 64，且链表的长度大于 8 的时候。

为什么是 8 呢？源码的注释也给出了答案。

```
*
* Because TreeNodes are about twice the size of regular nodes, we
* use them only when bins contain enough nodes to warrant use
* (see TREEIFY_THRESHOLD). And when they become too small (due to
* removal or resizing) they are converted back to plain bins. In
* usages with well-distributed user hashCodes, tree bins are
* rarely used. Ideally, under random hashCodes, the frequency of
* nodes in bins follows a Poisson distribution
* (http://en.wikipedia.org/wiki/Poisson\_distribution) with a
* parameter of about 0.5 on average for the default resizing
* threshold of 0.75, although with a large variance because of
* resizing granularity. Ignoring variance, the expected
* occurrences of list size k are (exp(-0.5) * pow(0.5, k) /
* factorial(k)). The first values are:
*
* 0: 0.60653066
* 1: 0.30326533
* 2: 0.07581633
* 3: 0.01263606
* 4: 0.00157952
* 5: 0.00015795
* 6: 0.00001316
* 7: 0.00000094
* 8: 0.00000006
* more: less than 1 in ten million
```

源码注释

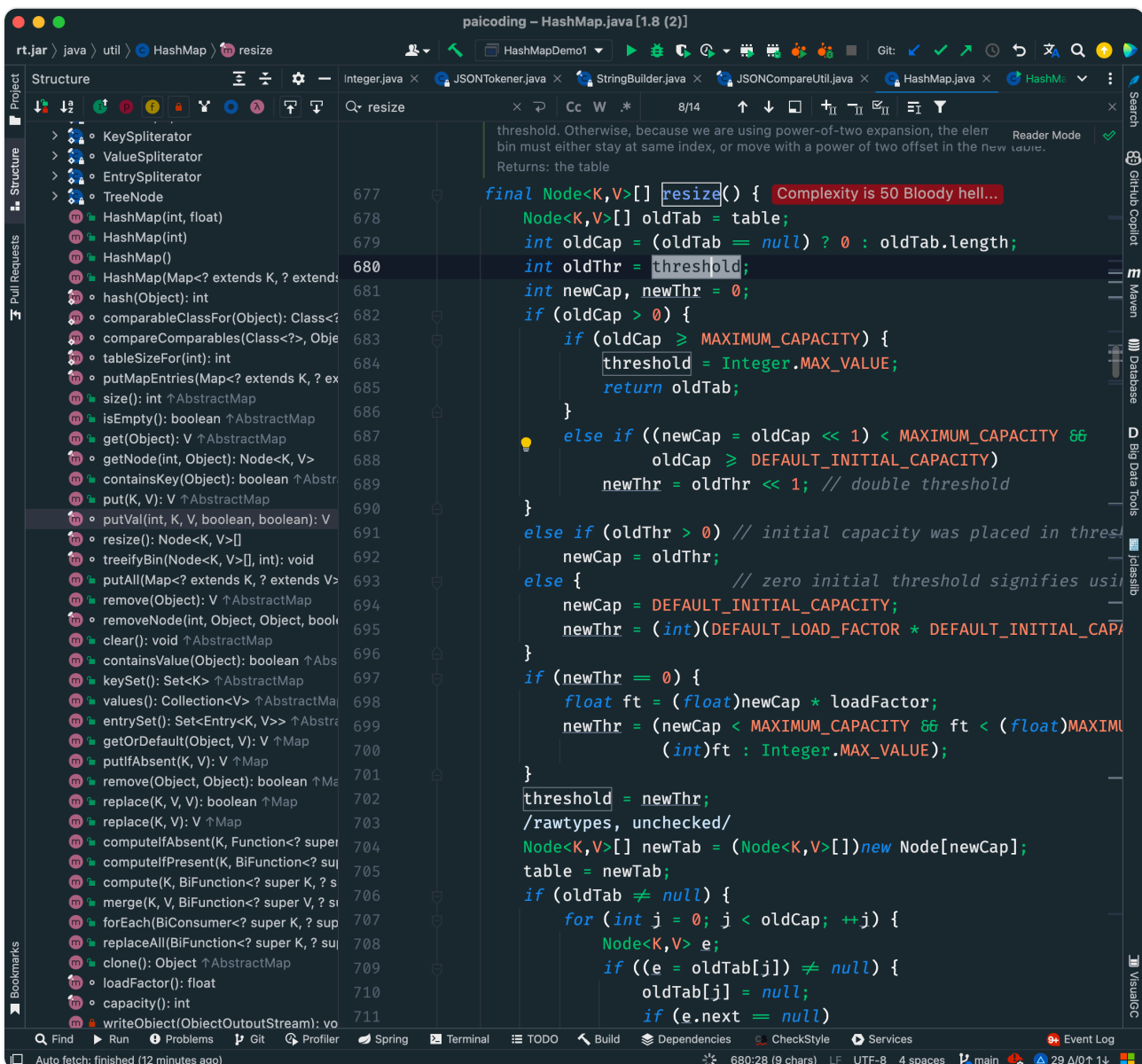
红黑树节点的大小大概是普通节点大小的两倍，所以转红黑树，牺牲了空间换时间，更多的是一种兜底的策略，保证极端情况下的查找效率。

阈值为什么要选 8 呢？和统计学有关。理想情况下，使用随机哈希码，链表里的节点符合泊松分布，出现节点个数的概率是递减的，节点个数为 8 的情况，发生概率仅为 0.00000006 。

至于红黑树转回链表的阈值为什么是 6，而不是 8？是因为如果这个阈值也设置成 8，假如发生碰撞，节点增减刚好在 8 附近，会发生链表和红黑树的不断转换，导致资源浪费。

20.扩容在什么时候呢？为什么扩容因子是 0.75？

HashMap 会在存储的键值对数量超过阈值（即容量 * 加载因子）时进行扩容。



默认的加载因子是 0.75，这意味着当 HashMap 填满了大约 75%的容量时，就会进行扩容。

```
static final int DEFAULT_INITIAL_CAPACITY = 1 << 4; // aka 16
```

java

默认的初始容量是 16，那就是大于 $16 \times 0.75 = 12$ 时，就会触发第一次扩容操作。

那么为什么选择了 0.75 作为 HashMap 的默认加载因子呢？

简单来说，这是对 空间 成本和 时间 成本平衡的考虑。

在 HashMap 中有这样一段注释：

```
*
* <p>As a general rule, the default load factor (.75) offers a good
* tradeoff between time and space costs. Higher values decrease the
* space overhead but increase the lookup cost (reflected in most of
* the operations of the <tt>HashMap</tt> class, including
* <tt>get</tt> and <tt>put</tt>). The expected number of entries in
* the map and its load factor should be taken into account when
* setting its initial capacity, so as to minimize the number of
* rehash operations. If the initial capacity is greater than the
* maximum number of entries divided by the load factor, no rehash
* operations will ever occur.
*
```

关于默认加载因子的注释

我们都知道，HashMap 的散列构造方式是 Hash 取余，加载因子决定元素个数达到多少时候扩容。

假如我们设的比较大，元素比较多，空位比较少的时候才扩容，那么发生哈希冲突的概率就增加了，查找的时间成本就增加了。

我们设的较小的话，元素比较少，空位比较多的时候就扩容了，发生哈希碰撞的概率就降低了，查找时间成本降低，但是就需要更多的空间去存储元素，空间成本就增加了。

1. [Java 面试指南（付费）](#) 收录的小米春招同学 K 一面面试原题：为什么是 2 次幂 到什么时候开始扩容 扩容机制流程

21.那扩容机制了解吗？

扩容时，HashMap 会创建一个新的数组，其容量是原数组容量的两倍。然后将键值对放到新计算出的索引位置上。一部分索引不变，另一部分索引为“原索引+旧容量”。

为了便于理解，我会结合 JDK7 和 JDK8 两个版本来讲。

在 JDK 7 中，定位元素位置的代码是这样的：

```
static int indexFor(int h, int length) {  
    // assert Integer.bitCount(length) == 1 : "length must be a non-zero  
    power of 2";  
    return h & (length-1);  
}
```

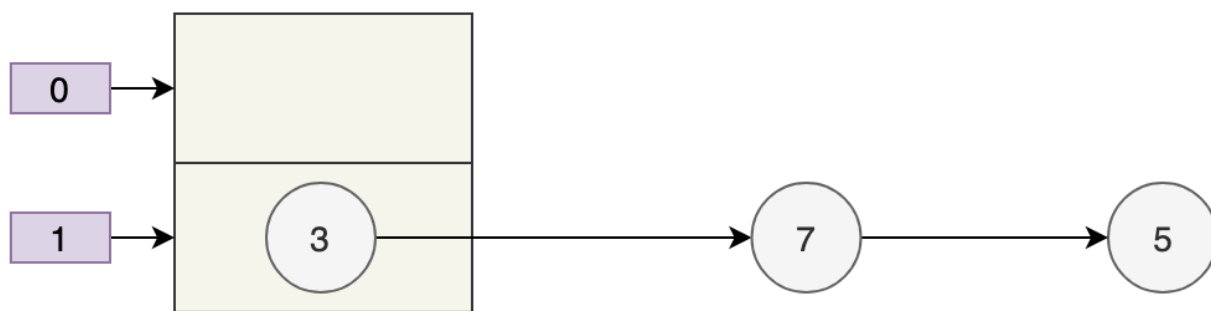
java

其实就相当于用键的哈希值和数组大小取模，也就是 `hashCode % table.length`。

那我们来假设：

- 数组 `table` 的长度为 2
- 键的哈希值为 3、7、5

取模运算后，键发生了哈希冲突，都到 `table[1]` 上了。那么扩容前就是这个样子。



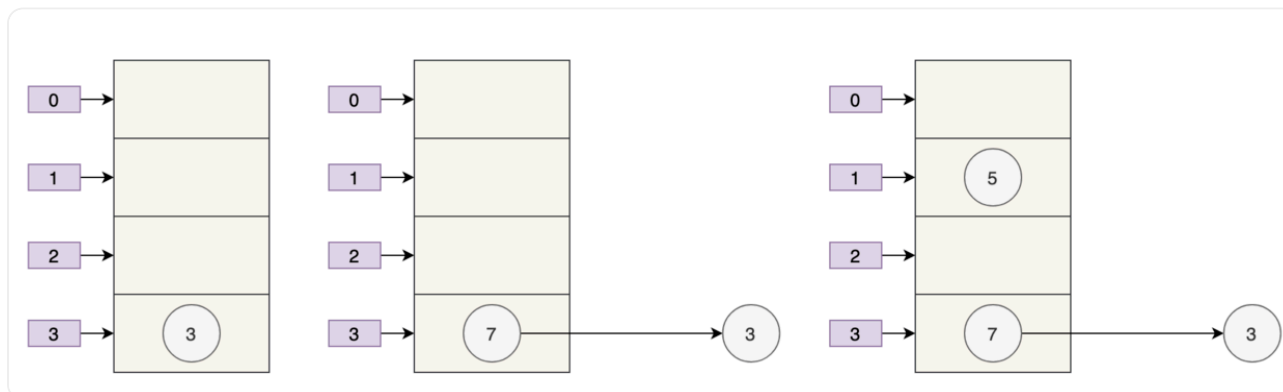
二哥的 Java 进阶之路：扩容前

数组的容量为 2，key 为 3、7、5 的元素在 `table[1]` 上，需要通过拉链法来解决哈希冲突。

假设负载因子 `loadFactor` 为 1，也就是当元素的个数大于 `table` 的长度时进行扩容。

扩容后的数组容量为 4。

- key 3 取模 ($3\%4$) 后是 3, 放在 `table[3]` 上。
- key 7 取模 ($7\%4$) 后是 3, 放在 `table[3]` 上的链表头部。
- key 5 取模 ($5\%4$) 后是 1, 放在 `table[1]` 上。



二哥的 Java 进阶之路：JDK7扩容后

7 跑到 3 的前面了，因为 JDK 7 使用的是头插法。

```
e.next = newTable[i];
```

java

同时，扩容后的 5 跑到了下标为 1 的位置。

最好的情况就是，扩容后的 7 在 3 的后面，5 在 7 的后面，保持原来的顺序。

JDK 8 完全扭转了这个局面，因为 JDK 8 的哈希算法进行了优化，当数组长度为 2 的幂次方时，能够很巧妙地解决 JDK 7 中遇到的问题。

JDK 8 的扩容代码如下所示：

```
Node<K,V>[] newTab = new Node[newCapacity];
for (int j = 0; j < oldTab.length; j++) {
    Node<K,V> e = oldTab[j];
    if (e != null) {
        int hash = e.hash;
        int newIndex = hash & (newCapacity - 1); // 计算在新数组中的位置
        // 将节点移动到新数组的对应位置
        newTab[newIndex] = e;
    }
}
```

java

新索引的计算方式是 $\text{hash} \& (\text{newCapacity} - 1)$ ，和 JDK 7 的 $\text{h} \& (\text{length}-1)$ 没什么大的差别，差别主要在 hash 方法上，JDK 8 是这样：

```
static final int hash(Object key) {  
    int h;  
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);  
}
```

java

过将键的 hashCode() 返回的 32 位哈希值与这个哈希值无符号右移 16 位的结果进行异或。

JDK 7 是这样：

```
final int hash(Object k) {  
    int h = hashSeed;  
    if (0 != h && k instanceof String) {  
        return sun.misc.Hashing.stringHash32((String) k);  
    }  
  
    h ^= k.hashCode();  
  
    // This function ensures that hashCodes that differ only by  
    // constant multiples at each bit position have a bounded  
    // number of collisions (approximately 8 at default load factor).  
    h ^= (h >>> 20) ^ (h >>> 12);  
    return h ^ (h >>> 7) ^ (h >>> 4);  
}
```

java

我们用 JDK 8 的哈希算法来计算一下哈希值，就会发现别有洞天。

假设扩容前的数组长度为 16 ($n-1$ 也就是二进制的 0000 1111, $1 \times 2^0 + 1 \times 2^1 + 1 \times 2^2 + 1 \times 2^3 = 1 + 2 + 4 + 8 = 15$)，key1 为 5（二进制为 0000 0101），key2 为 21（二进制为 0001 0101）。

- key1 和 $n-1$ 做 & 运算后为 0000 0101，也就是 5；
- key2 和 $n-1$ 做 & 运算后为 0000 0101，也就是 5。
- 此时哈希冲突了，用拉链法来解决哈希冲突。

现在，HashMap 进行了扩容，容量为原来的 2 倍，也就是 32 ($n-1$ 也就是二进制的 0001 1111, $1 \times 2^0 + 1 \times 2^1 + 1 \times 2^2 + 1 \times 2^3 + 1 \times 2^4 = 1 + 2 + 4 + 8 + 16 = 31$)。

- key1 和 $n-1$ 做 & 运算后为 0000 0101，也就是 5；
- key2 和 $n-1$ 做 & 运算后为 0001 0101，也就是 $21 = 5 + 16$ ，也就是数组扩容前的位置+原数组的长度。

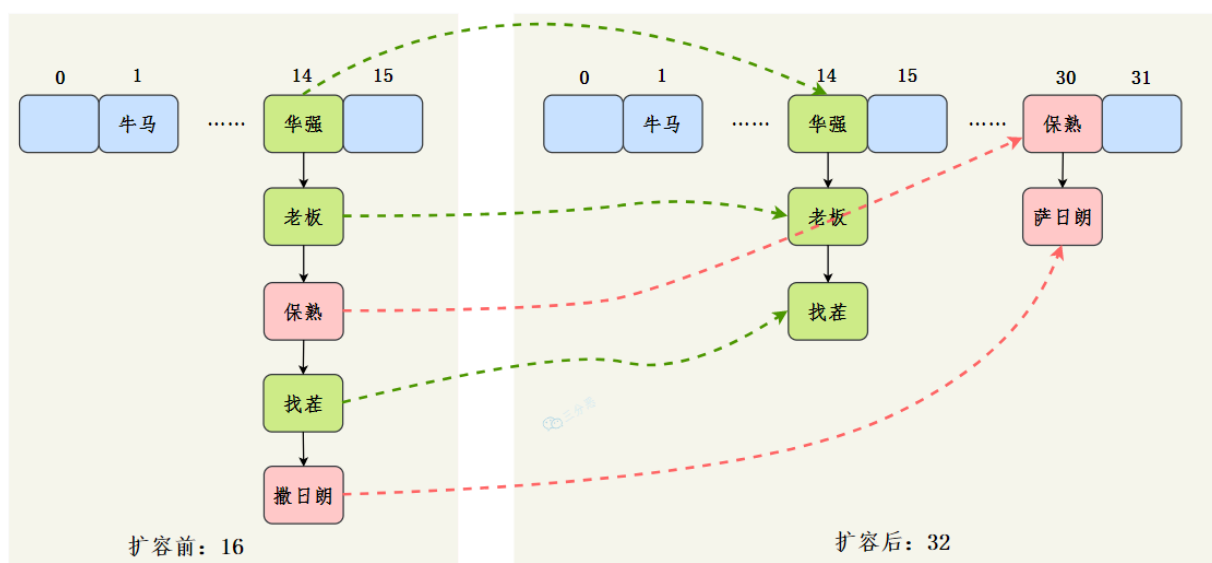
神奇吧？

$$\begin{array}{ccc} 0101=5 & \xrightarrow[16 \Rightarrow 2*16]{\text{resize}} & \begin{array}{l} 0 \ 0101 = 5 \\ 1 \ 0101 = 21 = 5 + 16 \end{array} \end{array}$$

原位置 原位置+oldCap

三分恶面渣逆袭：扩容位置变化

换句话说，在 JDK 8 的新 hash 算法下，数组扩容后的索引位置，要么就是原来的索引位置，要么就是“原索引+原来的容量”，遵循一定的规律。



三分恶面渣逆袭：扩容节点迁移示意图

当然了，这个功劳既属于新的哈希算法，也离不开 n 为 2 的整数次幂这个前提，这是它俩通力合作后的结果 $\text{hash} \& (\text{newCapacity} - 1)$ 。

那你想想扩容的时候每个节点都要进行位运算吗，如果我这个 **HashMap** 里面有几十万条数据，都要进行位运算吗？

在 JDK 8 的新 hash 算法下，数组扩容后的索引位置，要么就是原来的索引位置，要么就是“原索引+原来的容量”，遵循一定的规律。

具体来说，就是判断原哈希值的高位中新增的那一位是否为 1，如果是，该元素会被移动到原位置加上旧容量的位置；如果不是，则保持在原位置。

所以，尽管有几十万条数据，每个数据项的位置决定仅需要一次简单的位运算。位运算的计算速度非常快，因此，尽管扩容操作涉及遍历整个哈希表并对每个节点进行操作，但这部分操作的计算成本是相对较低的。

1. [Java 面试指南（付费）](#) 收录的小米春招同学 K 一面面试原题：为什么是 2 次幂 到什么时候开始扩容 扩容机制流程
2. [Java 面试指南（付费）](#) 收录的小米暑期实习同学 E 一面面试原题：说说 HashMap 的扩容机制，1.8 扩容具体实现
3. [Java 面试指南（付费）](#) 收录的奇安信面经同学 1 Java 技术一面面试原题：map 集合在使用时一般都需要写容量值？为什么要写？扩容机制？
4. [Java 面试指南（付费）](#) 收录的百度面经同学 1 文心一言 25 实习 Java 后端面试原题：hashmap 的底层实现原理、put()方法实现流程、扩容机制？

22.JDK 8 对 HashMap 主要做了哪些优化呢？为什么？

相比较 JDK 7，JDK 8 的 HashMap 主要做了四点优化：

①、底层数据结构由数组 + 链表改成了数组 + 链表或红黑树的结构。

原因：如果多个键映射到了同一个哈希值，链表会变得很长，在最坏的情况下，当所有的键都映射到同一个桶中时，性能会退化到 $O(n)$ ，而红黑树的时间复杂度是 $O(\log n)$ 。

②、链表的插入方式由头插法改为了尾插法。

原因：头插法虽然简单快捷，但扩容后容易改变原来链表的顺序。

③、扩容的时机由插入时判断改为插入后判断。

原因：可以避免在每次插入时都进行不必要的扩容检查，因为有可能插入后仍然不需要扩容。

④、优化了哈希算法。

JDK 7 进行了多次移位和异或操作来计算元素的哈希值。

```
final int hash(Object k) { Complexity is 11 You must be kidding
    int h = hashSeed;
    if (0 != h && k instanceof String) {
        return sun.misc.Hashing.stringHash32((String) k);
    }

    h ^= k.hashCode();

    // This function ensures that hashCodes that differ only by
    // constant multiples at each bit position have a bounded
    // number of collisions (approximately 8 at default load factor).
    h ^= (h >> 20) ^ (h >> 12);
    return h ^ (h >> 7) ^ (h >> 4);
}
```

二哥的 Java 进阶之路：JDK 7 的 hash 方法

JDK 8 优化了这个算法，只进行了一次异或操作，但仍然能有效地减少冲突。

```
static final int hash(Object key) { Complexity is 6 It's time to do something...
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >> 16);
}
```

二哥的 Java 进阶之路：JDK 8 的 hash 方法

并且能够保证扩容后，元素的新位置要么是原位置，要么是原位置加上旧容量大小。

1. [Java 面试指南（付费）](#) 收录的美团同学 2 优选物流调度技术 2 面面试题：HashMap 的内部结构，1.7 和 1.8 的区别，有什么改进

23.你能自己设计实现一个 HashMap 吗？

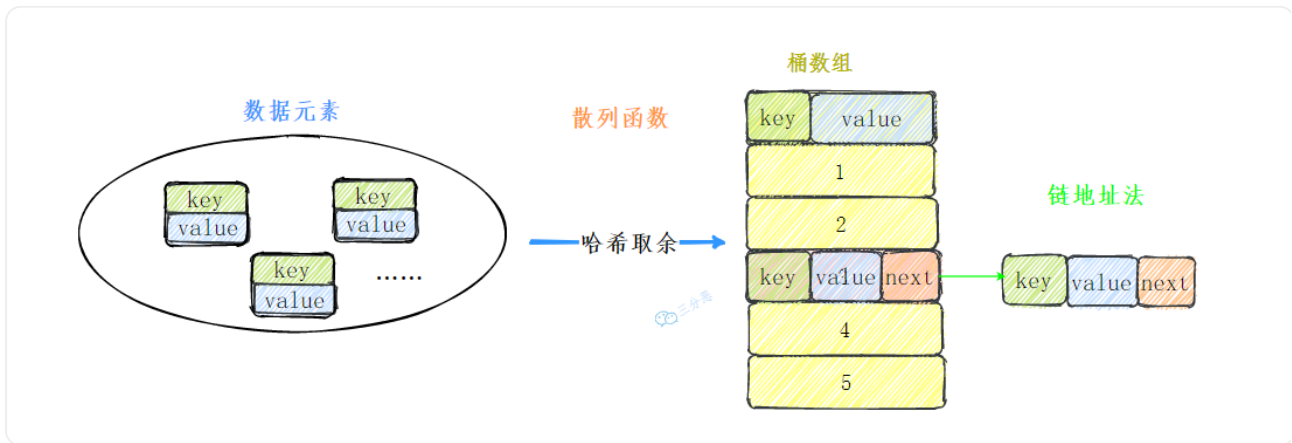
这道题快手常考。

不要慌，红黑树版咱们多半是写不出来，但是数组+链表版还是问题不大的，详细可见：[手写 HashMap，快手面试官直呼内行！](#)。

整体的设计：

- 散列函数：hashCode()+除留余数法
- 冲突解决：链地址法

- 扩容：节点重新 hash 获取位置



自定义HashMap整体结构

完整代码：

```
/**
 * @Author 三分恶
 * @Date 2021/11/21
 * @Description 自己实现HashMap
 */
public class ThirdHashMap<K, V> {

    /**
     * 节点类
     */
    @param <K>
    @param <V>
    */
    class Node<K, V> {
        //键值对
        private K key;
        private V value;

        //链表，后继
        private Node<K, V> next;

        public Node(K key, V value) {
            this.key = key;
            this.value = value;
        }

        public Node(K key, V value, Node<K, V> next) {
            this.key = key;
            this.value = value;
            this.next = next;
        }
    }

    //默认容量
    final int DEFAULT_CAPACITY = 16;
    //负载因子
    final float LOAD_FACTOR = 0.75f;
    //HashMap的大小
    private int size;
    //桶数组
    Node<K, V>[] buckets;

    /**
     * 无参构造器，设置桶数组默认容量
     */
    public ThirdHashMap() {
        buckets = new Node[DEFAULT_CAPACITY];
        size = 0;
    }

    /**
     * 有参构造器，指定桶数组容量
     */
}
```

```
    *
    * @param capacity
    */
    public ThirdHashMap(int capacity) {
        buckets = new Node[capacity];
        size = 0;
    }

    /**
     * 哈希函数，获取地址
     *
     * @param key
     * @return
     */
    private int getIndex(K key, int length) {
        //获取hash code
        int hashCode = key.hashCode();
        //和桶数组长度取余
        int index = hashCode % length;
        return Math.abs(index);
    }

    /**
     * put方法
     *
     * @param key
     * @param value
     * @return
     */
    public void put(K key, V value) {
        //判断是否需要扩容
        if (size >= buckets.length * LOAD_FACTOR) resize();
        putVal(key, value, buckets);
    }

    /**
     * 将元素存入指定的node数组
     *
     * @param key
     * @param value
     * @param table
     */
    private void putVal(K key, V value, Node<K, V>[] table) {
        //获取位置
        int index = getIndex(key, table.length);
        Node node = table[index];
        //插入的位置为空
        if (node == null) {
            table[index] = new Node<>(key, value);
            size++;
            return;
        }
        //插入位置不为空，说明发生冲突，使用链地址法，遍历链表
        while (node != null) {
            //如果key相同，就覆盖掉
            if ((node.key.hashCode() == key.hashCode())
                && (node.key == key || node.key.equals(key))) {
                node.value = value;
            }
        }
    }
}
```

```

        return;
    }
    node = node.next;
}
//当前key不在链表中，插入链表头部
Node newNode = new Node(key, value, table[index]);
table[index] = newNode;
size++;
}

/**
 * 扩容
 */
private void resize() {
    //创建一个两倍容量的桶数组
    Node<K, V>[] newBuckets = new Node[buckets.length * 2];
    //将当前元素重新散列到新的桶数组
    rehash(newBuckets);
    buckets = newBuckets;
}

/**
 * 重新散列当前元素
 *
 * @param newBuckets
 */
private void rehash(Node<K, V>[] newBuckets) {
    //map大小重新计算
    size = 0;
    //将旧的桶数组的元素全部刷到新的桶数组里
    for (int i = 0; i < buckets.length; i++) {
        //为空，跳过
        if (buckets[i] == null) {
            continue;
        }
        Node<K, V> node = buckets[i];
        while (node != null) {
            //将元素放入新数组
            putVal(node.key, node.value, newBuckets);
            node = node.next;
        }
    }
}

/**
 * 获取元素
 *
 * @param key
 * @return
 */
public V get(K key) {
    //获取key对应的地址
    int index = getIndex(key, buckets.length);
    if (buckets[index] == null) return null;
    Node<K, V> node = buckets[index];
    //查找链表
    while (node != null) {
        if ((node.key.hashCode() == key.hashCode())
            && (node.value != null && node.value.equals(key))) {
            return node.value;
        }
        node = node.next;
    }
    return null;
}

```

```
        && (node.key == key || node.key.equals(key))) {
            return node.value;
        }
        node = node.next;
    }
    return null;
}

/**
 * 返回HashMap大小
 *
 * @return
 */
public int size() {
    return size;
}
}
```

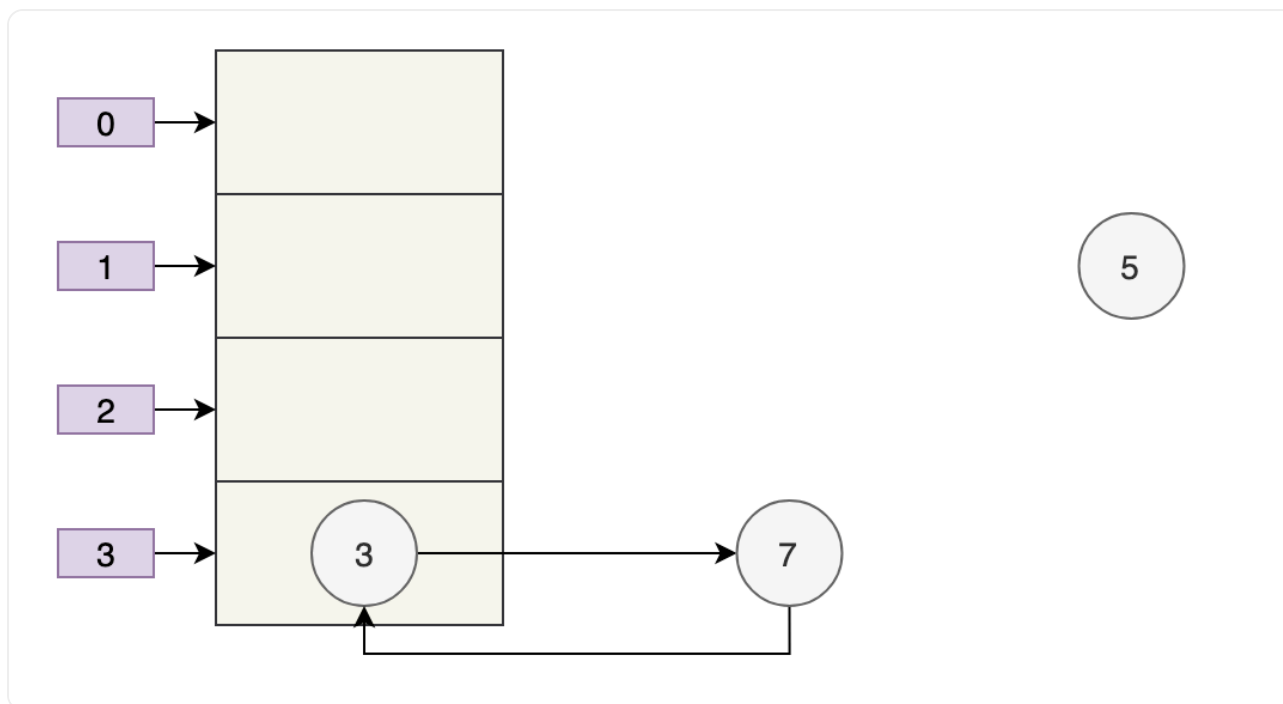
完整代码

24.HashMap 是线程安全的吗？多线程下会有什么问题？

推荐阅读：[HashMap 详解](#)

HashMap 不是线程安全的，主要有以下几个问题：

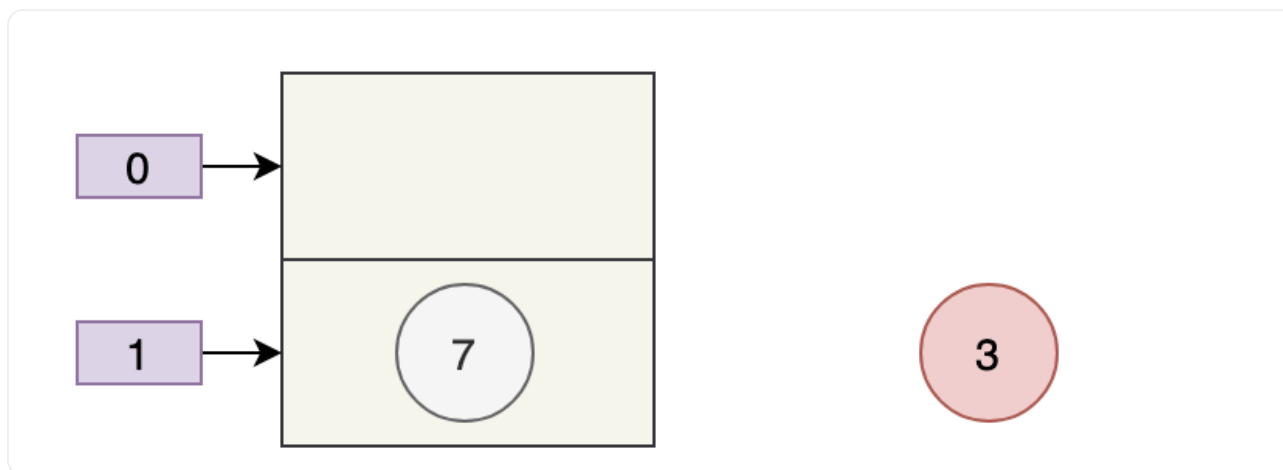
①、多线程下扩容会死循环。JDK1.7 中的 HashMap 使用的是头插法插入元素，在多线程的环境下，扩容的时候就有可能导致出现环形链表，造成死循环。



二哥的 Java 进阶之路

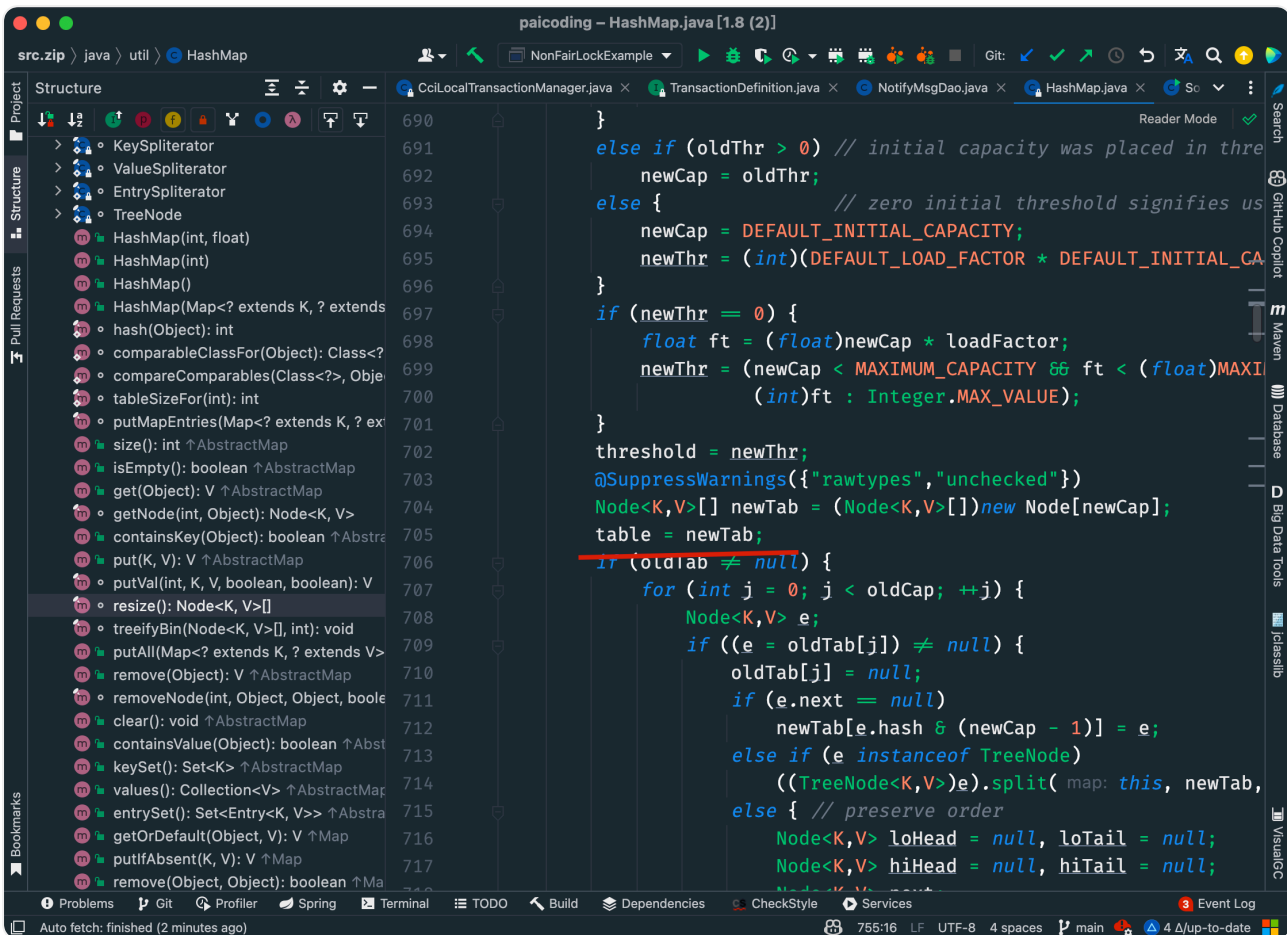
不过，JDK 8 时已经修复了这个问题，扩容时会保持链表原来的顺序。

②、多线程的 put 可能会导致元素的丢失。因为计算出来的位置可能会被其他线程的 put 覆盖。本来哈希冲突是应该用链表的，但多线程时由于没有加锁，相同位置的元素可能就被干掉了。



二哥的 Java 进阶之路

③、put 和 get 并发时，可能导致 get 为 null。线程 1 执行 put 时，因为元素个数超出阈值而导致出现扩容，线程 2 此时执行 get，就有可能出现这个问题。



二哥的 Java 进阶之路：源码截图

因为线程 1 执行完 `table = newTab` 之后，线程 2 中的 `table` 此时也发生了变化，此时去 `get` 的时候当然会 `get` 到 `null` 了，因为元素还没有转移。

1. [Java 面试指南（付费）](#) 收录的华为 OD 原题：HashMap 是线程安全的吗？
2. [Java 面试指南（付费）](#) 收录的华为面经同学 8 技术二面面试原题：HashMap 是线程安全的吗？
3. [Java 面试指南（付费）](#) 收录的字节跳动面经同学 9 飞书后端技术一面面试原题：HashMap 为什么不安全，如何改进，以及 ConcurrentHashMap
4. [Java 面试指南（付费）](#) 收录的腾讯云智面经同学 16 一面面试原题：HashMap 的底层实现，它为什么是线程不安全的？

25.有什么办法能解决 HashMap 线程不安全的问题呢？

在 Java 中，有 3 种线程安全的 Map 实现，最常用的是 [ConcurrentHashMap](#) 和 `Collections.synchronizedMap(Map)` 包装器。

Hashtable 也是线程安全的，但它的使用已经不再推荐使用，因为 ConcurrentHashMap 提供了更高的并发性和性能。

①、Hashtable 是直接和方法上加 [synchronized 关键字](#)，比较粗暴。

从 Java 2 平台 v1.2 开始，该类被修改为实现接口 `Map`，使其成为 `Java 集合框架` 的成员。与新的集合实现不同，`Hashtable` 它是同步的。如果不需要线程安全实现，建议使用 `HashMap` 代替 `Hashtable`。如果需要线程安全的高并发实现，则建议使用 `java.util.concurrent.ConcurrentHashMap` 代替 `Hashtable`。

因为： JDK1.0

另请参见： `Object.equals(Object)`,
`Object.hashCode()`,
`rehash()`,
`Collection`,
`Map`,
`HashMap`,
`TreeMap`

作者： 亚瑟·范霍夫、乔什·布洛赫、尼尔·格夫特

```
public class Hashtable<K,V> Complexity is 28 Bloody hell...  
    extends Dictionary<K,V>  
    implements Map<K,V>, Cloneable, java.io.Serializable {
```

二哥的 Java 进阶之路：Hashtable

②、`Collections.synchronizedMap` 返回的是 [Collections](#) 工具类的内部类。



③、ConcurrentHashMap 在 JDK 7 中使用分段锁，在 JDK 8 中使用了 CAS (Compare-And-Swap) + synchronized 关键字，性能得到进一步提升。

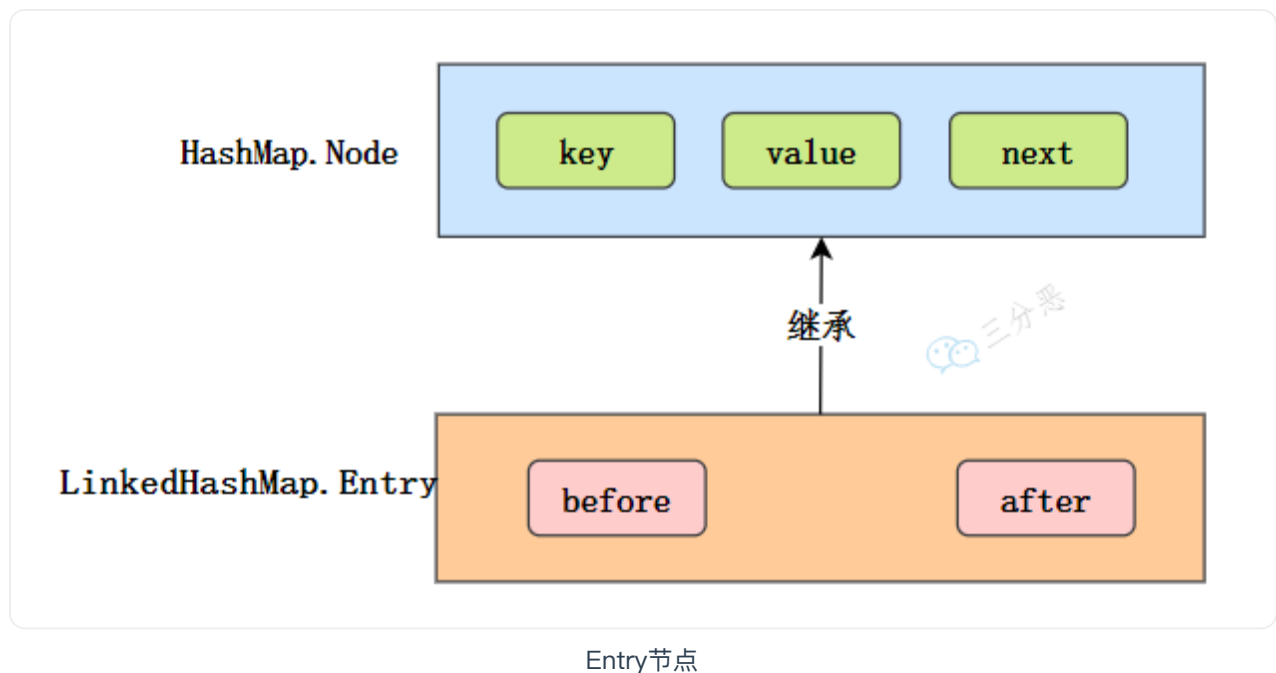


1. [Java 面试指南（付费）](#) 收录的小米春招同学 K 一面面试原题：有哪些线程安全的 map，ConcurrentHashMap 怎么保证线程安全的，为什么比 hashTable 效率高
2. [Java 面试指南（付费）](#) 收录的华为面经同学 8 技术二面面试原题：Java 中的线程安全的集合是什么？
3. [Java 面试指南（付费）](#) 收录的字节跳动面经同学 9 飞书后端技术一面面试原题：HashMap 为什么不安全，如何改进，以及 ConcurrentHashMap
4. [Java 面试指南（付费）](#) 收录的腾讯云智面经同学 16 一面面试原题：知道哪些线程安全的集合类型？
5. [Java 面试指南（付费）](#) 收录的招商银行面经同学 6 招银网络科技面试原题：线程不安全的集合变成线程安全的方法？

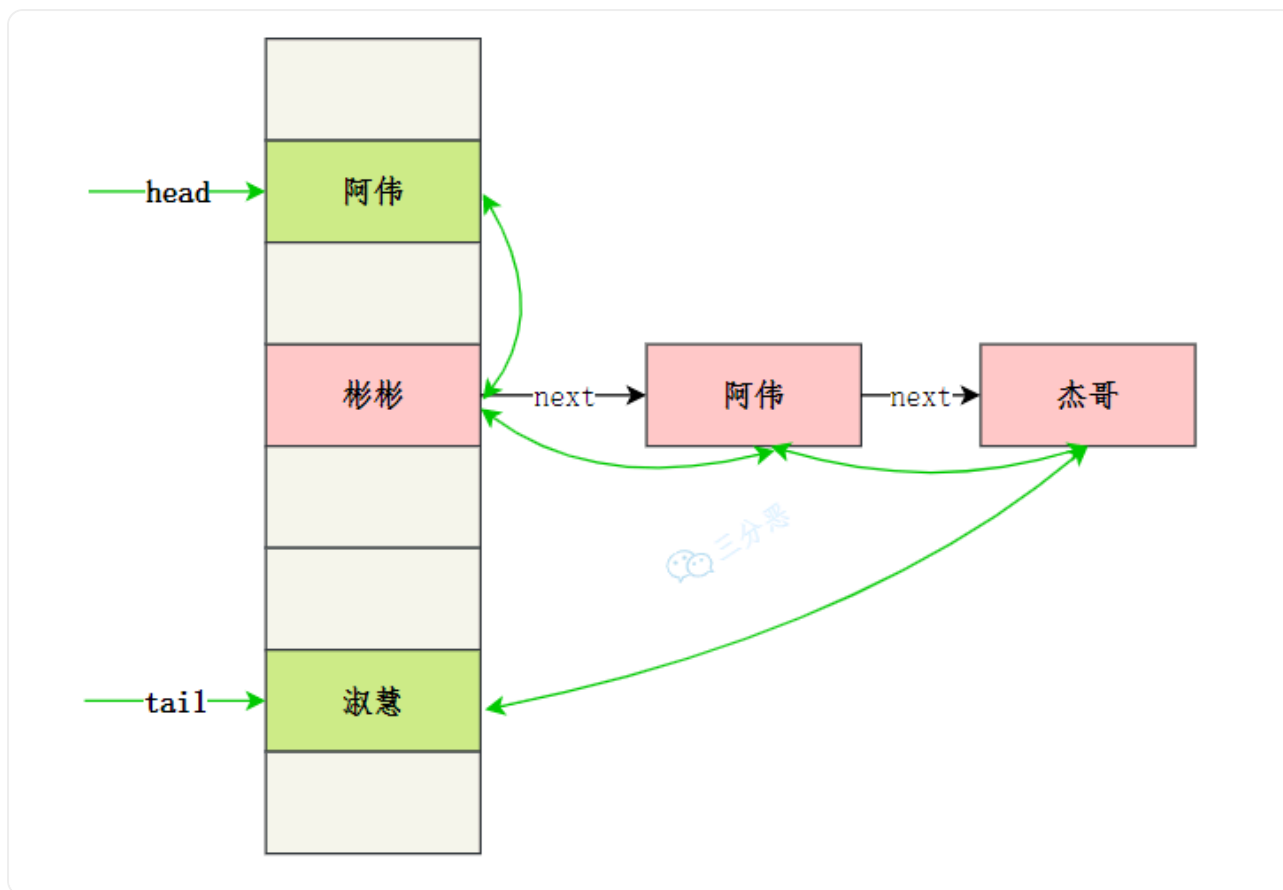
HashMap 是无序的，根据 hash 值随机插入。如果想使用有序的 Map，可以使用 LinkedHashMap 或者 TreeMap。

27.讲讲 LinkedHashMap 怎么实现有序的？

LinkedHashMap 维护了一个双向链表，有头尾节点，同时 LinkedHashMap 节点 Entry 内部除了继承 HashMap 的 Node 属性，还有 before 和 after 用于标识前置节点和后置节点。



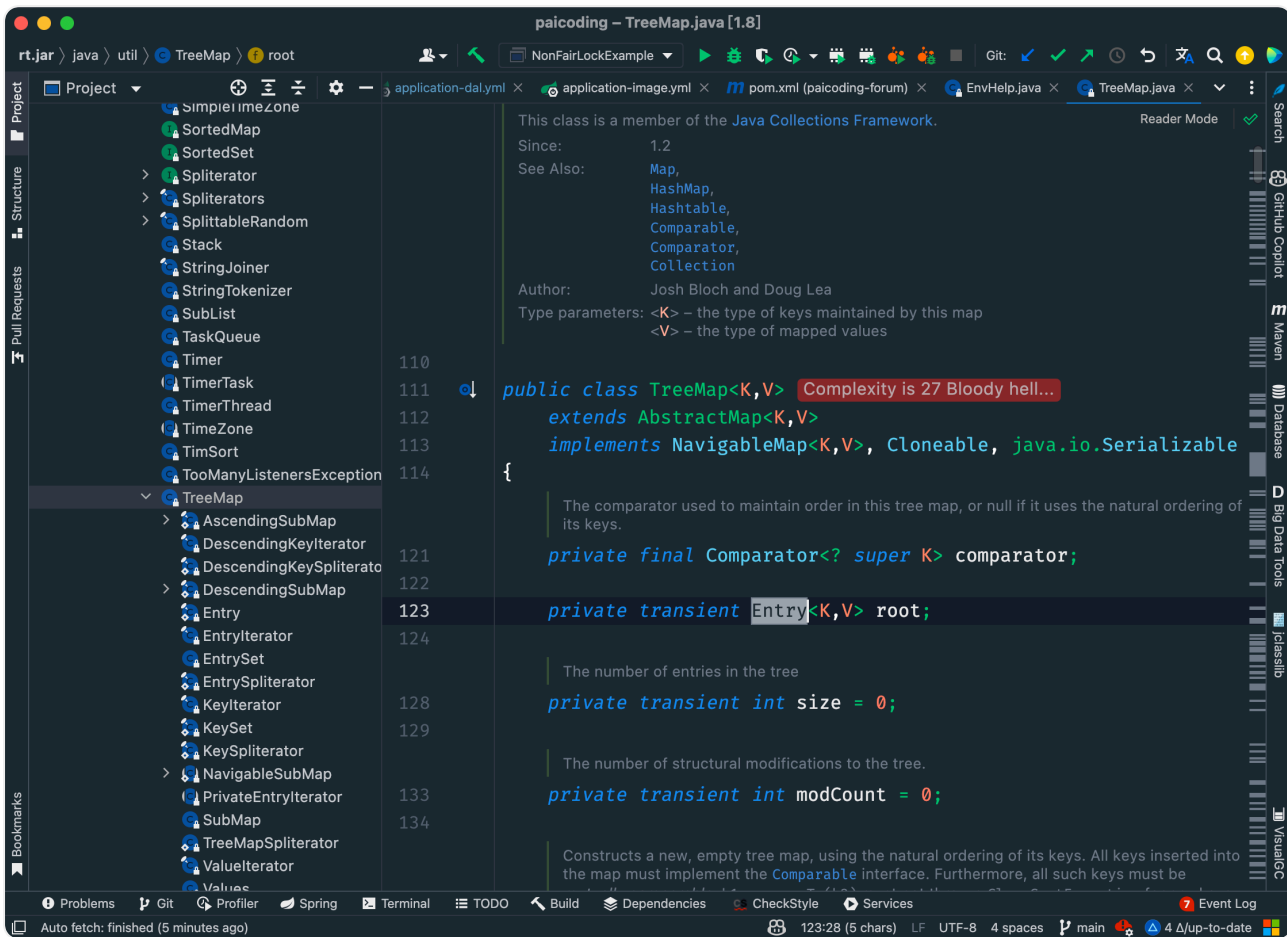
可以实现按插入的顺序或访问顺序排序。



LinkedHashMap实现原理

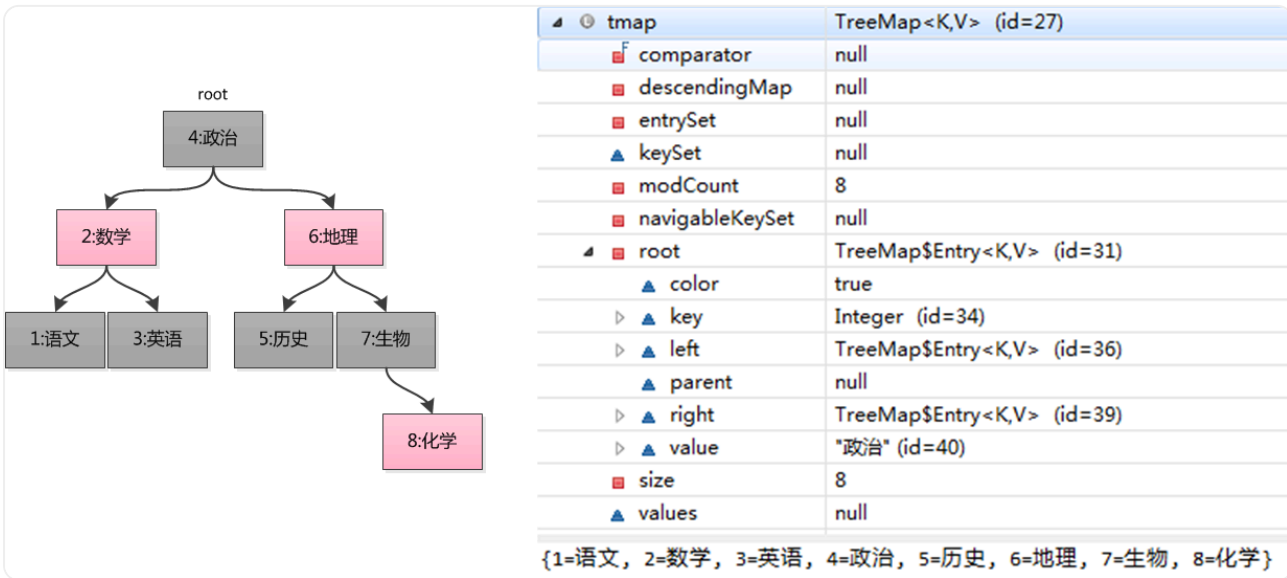
28.讲讲 TreeMap 怎么实现有序的？

TreeMap 通过 key 的比较器来决定元素的顺序，如果没有指定比较器，那么 key 必须实现 [Comparable 接口](#)。



二哥的 Java 进阶之路

TreeMap 的底层是红黑树，红黑树是一种自平衡的二叉查找树，每个节点都大于其左子树中的任何节点，小于其右子节点树种的任何节点。



三分恶面渣逆袭：TreeMap

插入或者删除元素时通过旋转和着色来保持树的平衡。

查找的时候通过从根节点开始，利用二叉查找树的性质，逐步向左或者右子树递归查找，直到找到目标元素。

29.TreeMap 和 HashMap 的区别

①、HashMap 是基于数组+链表+红黑树实现的，put 元素的时候会先计算 key 的哈希值，然后通过哈希值计算出数组的索引，然后将元素插入到数组中，如果发生哈希冲突，会使用链表来解决，如果链表长度大于 8，会转换为红黑树。

get 元素的时候同样会先计算 key 的哈希值，然后通过哈希值计算出数组的索引，如果遇到链表或者红黑树，会通过 key 的 equals 方法来判断是否是要找的元素。

②、TreeMap 是基于红黑树实现的，put 元素的时候会先判断根节点是否为空，如果为空，直接插入到根节点，如果不为空，会通过 key 的比较器来判断元素应该插入到左子树还是右子树。

get 元素的时候会通过 key 的比较器来判断元素的位置，然后递归查找。

由于 HashMap 是基于哈希表实现的，所以在没有发生哈希冲突的情况下，HashMap 的查找效率是 $O(1)$ 。适用于查找操作比较频繁的场景。

而 TreeMap 是基于红黑树实现的，所以 TreeMap 的查找效率是 $O(\log n)$ 。并且保证了元素的顺序，因此适用于需要大量范围查找或者有序遍历的场景。

1. [Java 面试指南（付费）](#) 收录的美团面经同学 16 暑期实习一面面试原题：知道哪些集合，讲讲 HashMap 和 TreeMap 的区别，讲讲两者应用场景的区别

Set

30.讲讲 HashSet 的底层实现？

HashSet 其实是由 HashMap 实现的，只不过值由一个固定的 Object 对象填充，而键用于操作。

```
public class HashSet<E>
    extends AbstractSet<E>
    implements Set<E>, Cloneable, java.io.Serializable
{
    static final long serialVersionUID = -5024744406713321676L;
```

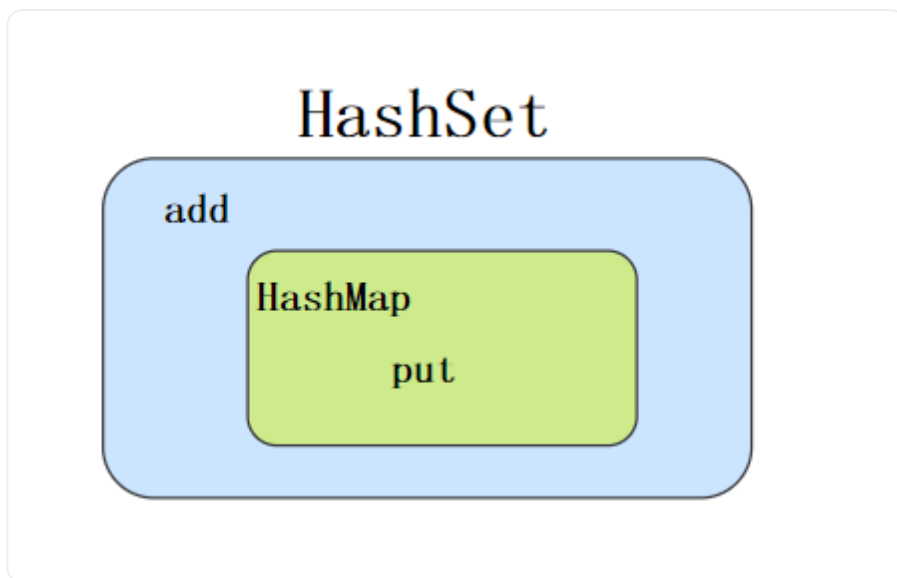
```
private transient HashMap<E, Object> map;  
// Dummy value to associate with an Object in the backing Map  
private static final Object PRESENT = new Object();  
// .....  
}
```

实际开发中，HashSet 并不常用，比如，如果我们需要按照顺序存储一组元素，那么 ArrayList 和 LinkedList 可能更适合；如果我们需要存储键值对并根据键进行查找，那么 HashMap 可能更适合。

HashSet 主要用于去重，比如，我们需要统计一篇文章中有多少个不重复的单词，就可以使用 HashSet 来实现。

```
// 创建一个 HashSet 对象  
HashSet<String> set = new HashSet<>();  
  
// 添加元素  
set.add("沉默");  
set.add("王二");  
set.add("陈清扬");  
set.add("沉默");  
  
// 输出 HashSet 的元素个数  
System.out.println("HashSet size: " + set.size()); // output: 3  
  
// 遍历 HashSet  
for (String s : set) {  
    System.out.println(s);  
}
```

HashSet 会自动去重，因为它是用 HashMap 实现的，HashMap 的键是唯一的（哈希值），相同键的值会覆盖掉原来的值，于是第二次 set.add("沉默") 的时候就覆盖了第一次的 set.add("沉默")。



三分恶面渣逆袭：HashSet套娃

HashSet 和 ArrayList 的区别

- ArrayList 是基于动态数组实现的，HashSet 是基于 HashMap 实现的。
- ArrayList 允许重复元素和 null 值，可以有多个相同的元素；HashSet 保证每个元素唯一，不允许重复元素，基于元素的 hashCode 和 equals 方法来确定元素的唯一性。
- ArrayList 保持元素的插入顺序，可以通过索引访问元素；HashSet 不保证元素的顺序，元素的存储顺序依赖于哈希算法，并且可能随着元素的添加或删除而改变。

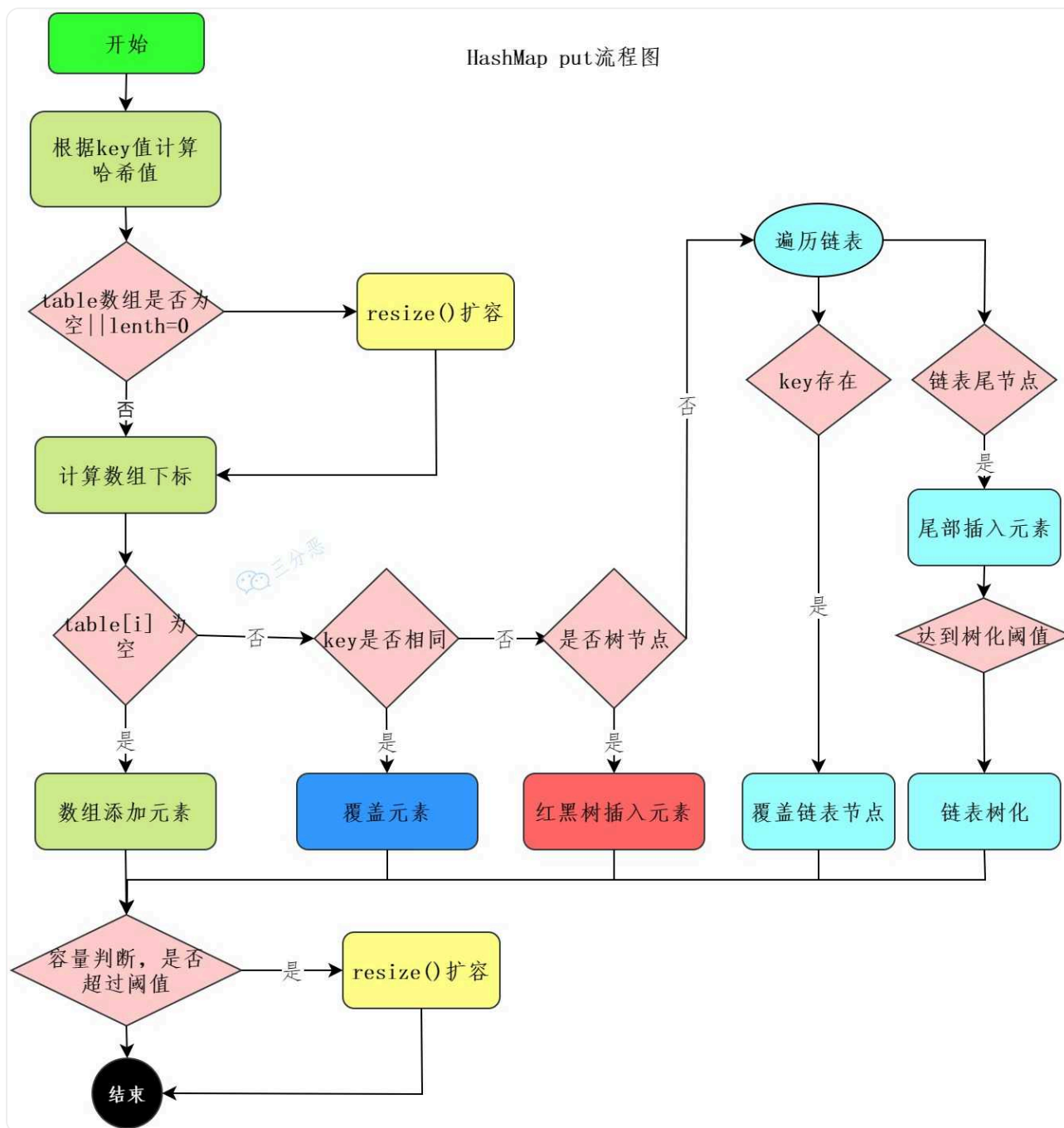
HashSet 怎么判断元素重复，重复了是否 put

HashSet 的 add 方法是通过调用 HashMap 的 put 方法实现的：

```
public boolean add(E e) {  
    return map.put(e, PRESENT) == null;  
}
```

java

所以 HashSet 判断元素重复的逻辑底层依然是 HashMap 的底层逻辑：



三分恶面渣逆袭：HashMap插入数据流程图

HashMap 在插入元素时，通常需要三步：

第一步，通过 hash 方法计算 key 的哈希值。

```
static final int hash(Object key) {  
    int h;  
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);  
}
```

java

第二步，数组进行第一次扩容。

```
if ((tab = table) == null || (n = tab.length) == 0)
    n = (tab = resize()).length;
```

java

第三步，根据哈希值计算 key 在数组中的下标，如果对应下标正好没有存放数据，则直接插入。

```
if ((p = tab[i = (n - 1) & hash]) == null)
    tab[i] = newNode(hash, key, value, null);
```

java

如果对应下标已经有数据了，就需要判断是否为相同的 key，是则覆盖 value，否则需要判断是否为树节点，是则向树中插入节点，否则向链表中插入数据。

```
else {
    Node<K,V> e; K k;
    if (p.hash == hash &&
        ((k = p.key) == key || (key != null && key.equals(k))))
        e = p;
    else if (p instanceof TreeNode)
        e = ((TreeNode<K,V>)p).putTreeVal(this, tab, hash, key, value);
    else {
        for (int binCount = 0; ; ++binCount) {
            if ((e = p.next) == null) {
                p.next = newNode(hash, key, value, null);
                if (binCount >= TREEIFY_THRESHOLD - 1) // -1 for 1st
                    treeifyBin(tab, hash);
                break;
            }
            if (e.hash == hash &&
                ((k = e.key) == key || (key != null && key.equals(k))))
                break;
            p = e;
        }
    }
}
```

java

也就是说，HashSet 通过元素的哈希值来判断元素是否重复，如果重复了，会覆盖原来的值。

```
if (e != null) { // existing mapping for key
    V oldValue = e.value;
    if (!onlyIfAbsent || oldValue == null)
        e.value = value;
    afterNodeAccess(e);
    return oldValue;
}
```

java

1. [Java 面试指南（付费）](#) 收录的京东同学 10 后端实习一面的原题：HashSet 和 ArrayList 的区别
2. [Java 面试指南（付费）](#) 收录的京东同学 10 后端实习一面的原题：HashSet 怎么判断元素重复，重复了是否 put

图文详解 29 道 Java 集合框架面试高频题，这次吊打面试官，我觉得稳了（手动 dog）。整理：沉默王二，戳[转载链接](#)，作者：三分恶，戳[原文链接](#)。

没有什么使我停留——除了目的，纵然岸旁有玫瑰、有绿荫、有宁静的港湾，我是不系之舟。

系列内容：

- [面渣逆袭 Java SE 篇](#) 👍
- [面渣逆袭 Java 集合框架篇](#) 👍
- [面渣逆袭 Java 并发编程篇](#) 👍
- [面渣逆袭 JVM 篇](#) 👍
- [面渣逆袭 Spring 篇](#) 👍
- [面渣逆袭 Redis 篇](#) 👍
- [面渣逆袭 MyBatis 篇](#) 👍
- [面渣逆袭 MySQL 篇](#) 👍
- [面渣逆袭操作系统篇](#) 👍
- [面渣逆袭计算机网络篇](#) 👍
- [面渣逆袭 RocketMQ 篇](#) 👍
- [面渣逆袭分布式篇](#) 👍
- [面渣逆袭微服务篇](#) 👍
- [面渣逆袭设计模式篇](#) 👍
- [面渣逆袭 Linux 篇](#) 👍

GitHub 上标星 10000+ 的开源知识库《[二哥的 Java 进阶之路](#)》第一版 PDF 终于来了！包括 Java 基础语法、数组&字符串、OOP、集合框架、Java IO、异常处理、Java 新特性、网络编程、NIO、并发编程、JVM 等等，共计 32 万余字，500+张手绘图，可以说是通俗易懂、风趣幽默.....详情戳：[太赞了，GitHub 上标星 10000+ 的 Java 教程](#)

微信搜 沉默王二 或扫描下方二维码关注二哥的原创公众号沉默王二，回复 222 即可免费领取。



Copyright © 2024 三分恶